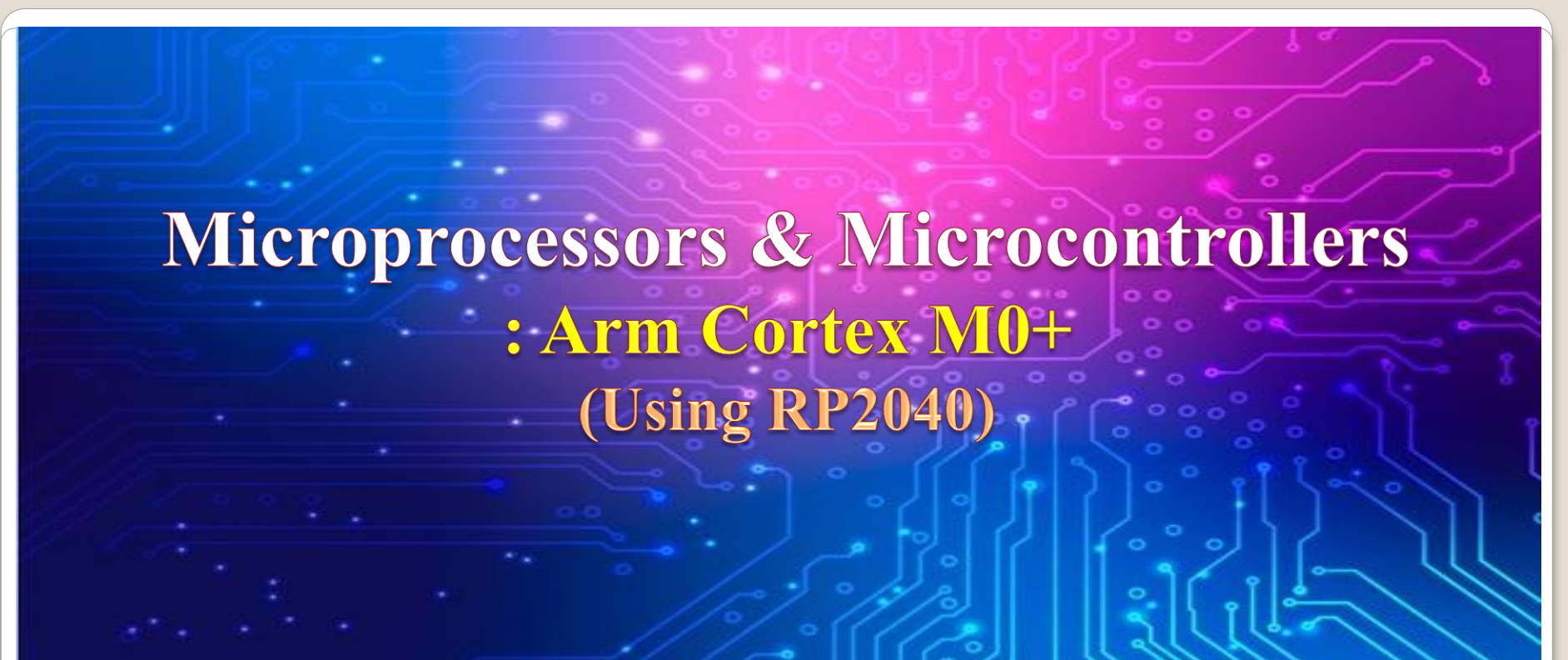


Embedded System and Microcontroller



Microprocessors & Microcontrollers

: Arm Cortex M0+

(Using RP2040)

Unit2

Microprocessors Vs Microcontrollers

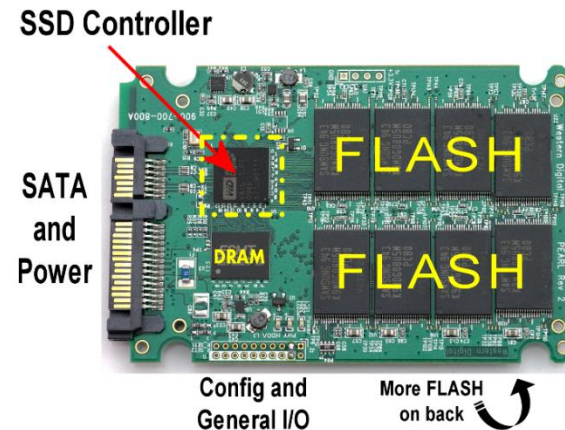
Mouli Sankaran

Different Levels of Implementation

Only CPU core and L1 cache inside the chip.
Outside: L2 caches, RAM memory, peripherals



CPU core and cache inside the chip
Outside: RAM/FLASH memory, peripherals



SSD: Solid State Device (Flash memory)

SATA: Hard-disk interface, used early in PCs.

SATA: Serial ATA

ATA: Advanced Technology Interface

Microcontroller:

CPU core, SRAM, EEPROM,

FLASH memory and

Peripherals **all inside the chip**

Only I/O pins come out of the chip

μ P Vs μ C

Microprocessor



No. of Pins: **423**

MCUs have a simpler core logic having space to put memory, peripherals inside, resulting in a simpler, low power system, with less number of pins

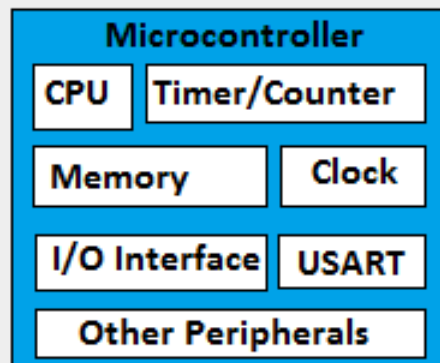
Microcontroller



No. of Pins: **32**

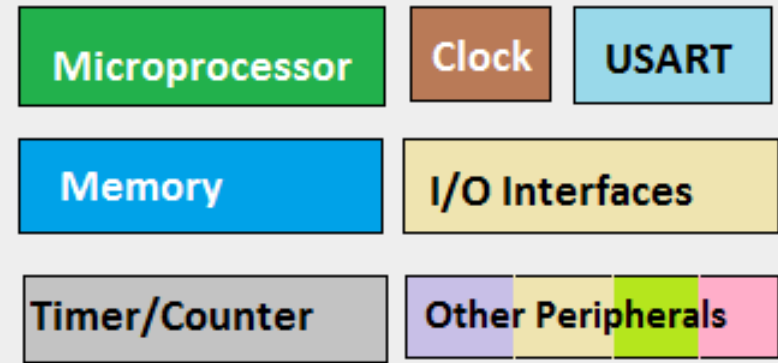
μ C

One chip



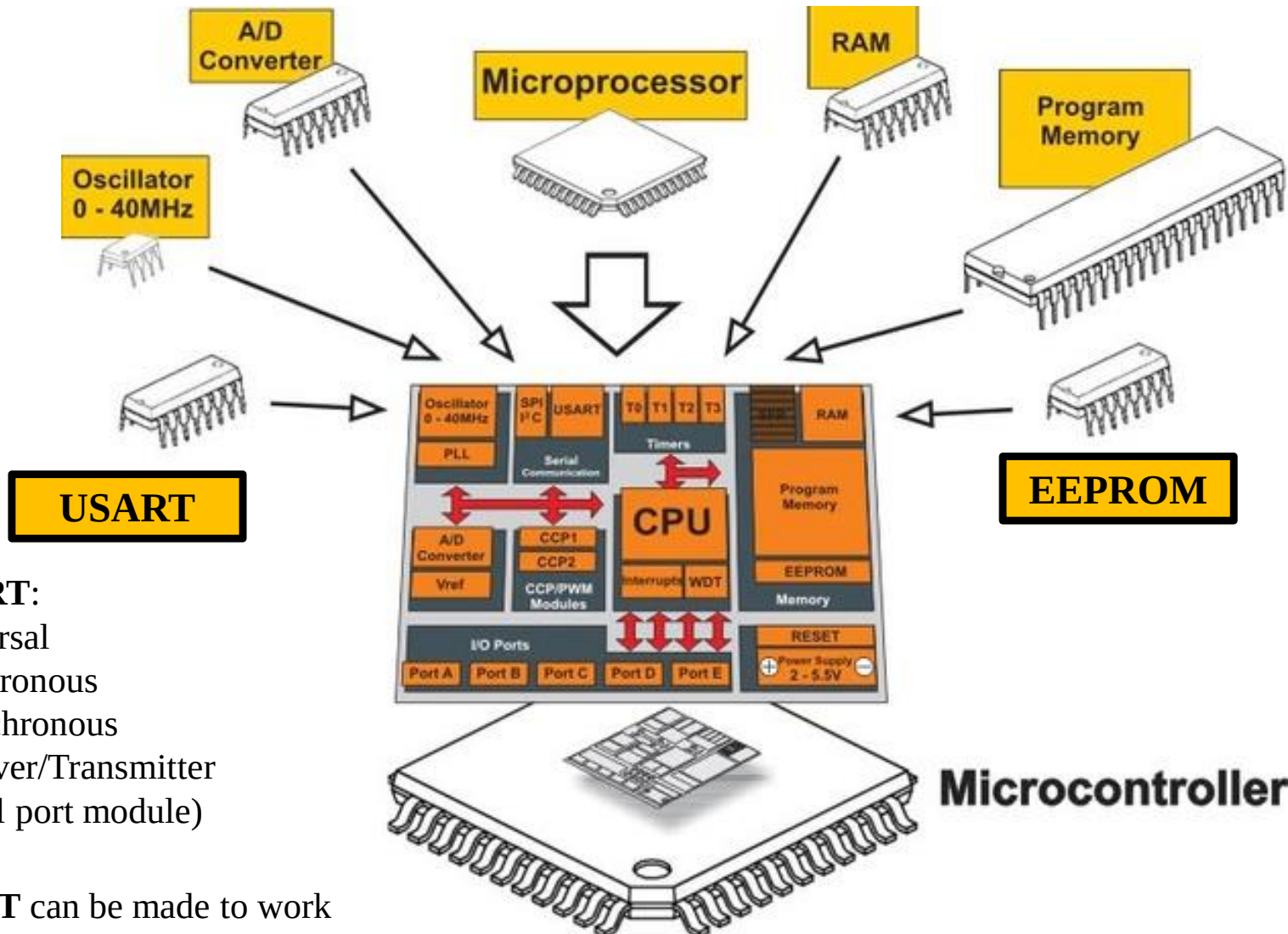
μ P

Many different chips



With **μ P**, many chips are required to build a system. Memory is outside because μ P core is of complex design having no space to accommodate memory inside the chip.

Inside a Microcontroller



USART:
Universal
Synchronous
Asynchronous
Receiver/Transmitter
(Serial port module)

USART can be made to work
in the Synchronous mode also

Quiz 1: Why do MCUs consume less power?

- Choose **all the valid reasons** below:
 - A. Simpler processor core design with less number of instructions
 - B. Both code and data memories are built into the chip
 - C. Most of the peripherals are built into the chip
 - D. Most of the time, the processor accesses internal modules than driving signals over the pins to access external modules

Answer: **A, B, C, D**

Note: When the CPU needs to access the external modules, it needs to drive signals over long lines requiring more power. Since MCUs most of the time access internal modules, they consume less power compared to microprocessors, because memory and most of the peripherals are outside the microprocessor chips.

Other Micro Controllers (MCUs)



ARM7



ARM9



TI MSP430



PIC
microcont...



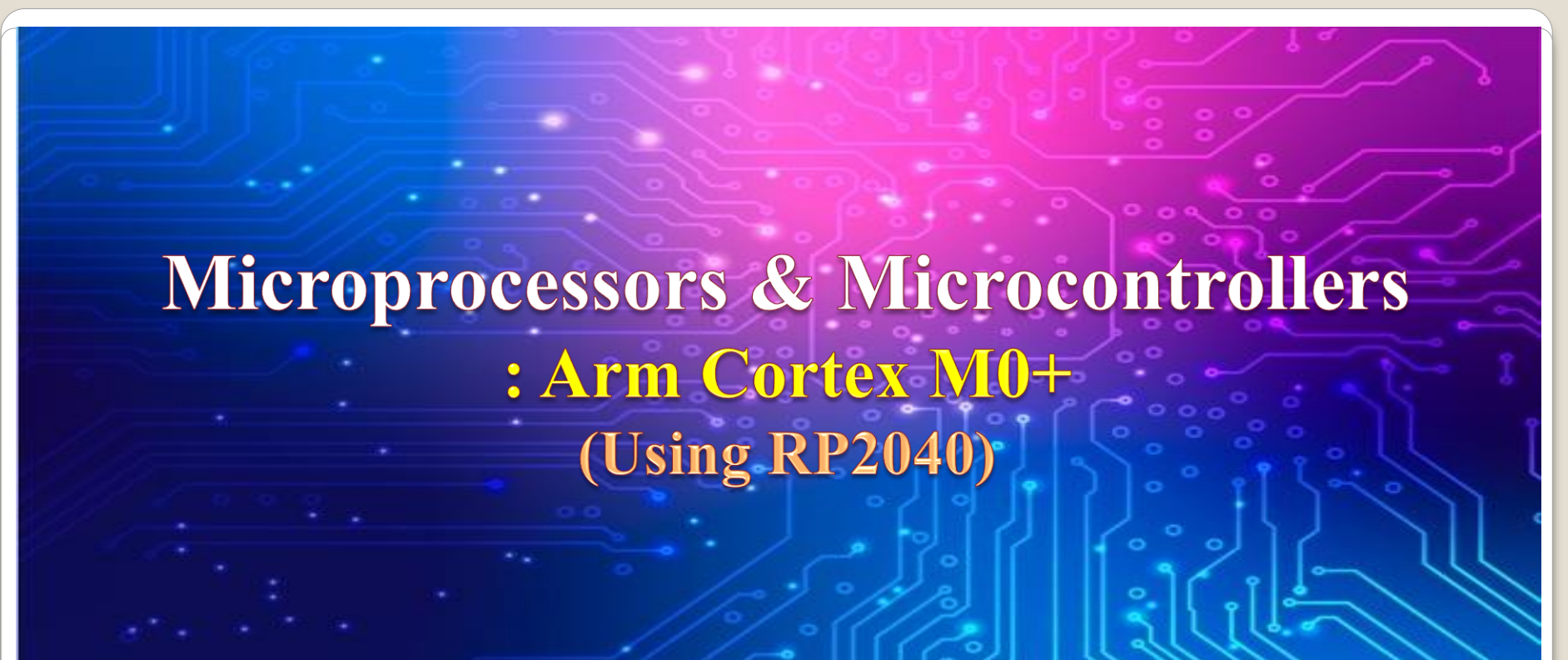
STM32

Here are some of popular microcontrollers used for industry needs.

- ARM controllers (ARM 7, ARM 9 ARM 11)
- MSP430 controllers. **16-bit μ C from TI**
- PIC microcontroller. **8 to 32-bit MCUs from Microchip Technology**
- Renesas family. **Renesas Electronics, Tokyo, Japan**
- STM electronics (STM 32, STR9 series) **STMircroelectronics, Switzerland.**



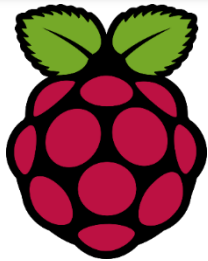
8051: μ C from Intel



Microprocessors & Microcontrollers **: Arm Cortex M0+** **(Using RP2040)**

Introduction to Raspberry Pi Family

Raspberry Pi Family (Introduction)



Raspberry Pi Logo

The **Raspberry Pi Foundation** is a **British charity and company** founded in **2009**, to promote the **study of basic computer science** in **schools**, and is responsible for developing the **Raspberry Pi single-board computers**.

Raspberry Pi

A few Boards of Raspberry Pi family

Raspberry Pi Zero
Released in Nov 2015



Raspberry Pi 3 Model B+
Released in 2016



Raspberry Pi 4 B
Released in Jun 2019



- **Raspberry Pi Pico** was released in **January 2021** with a retail price of **\$4**.
- It was Raspberry Pi's first board, based on a single **ARM microcontroller** chip designed in the UK.
- It is programmable in MicroPython, CircuitPython, and C/C++.
- It is designed for physical computing, similar in concept to an Arduino.

Raspberry Pi Pico (RP2040)

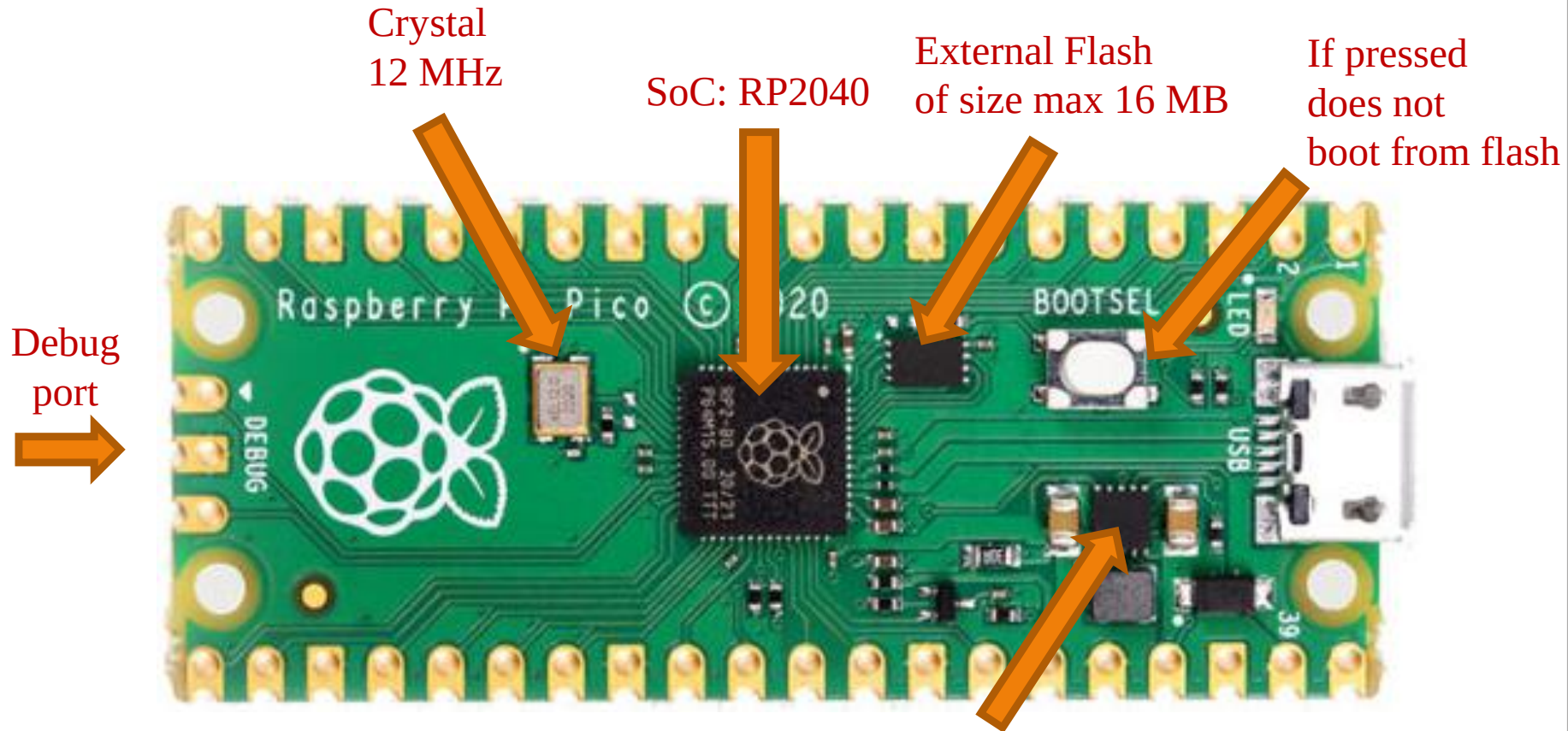
Front side



Back side



Raspberry Pi Pico (based on RP2040): Layout



SoC: System on a Chip

SMPS: Switched Mode Power Supply

**3.3 V Buck Boost
SMPS**

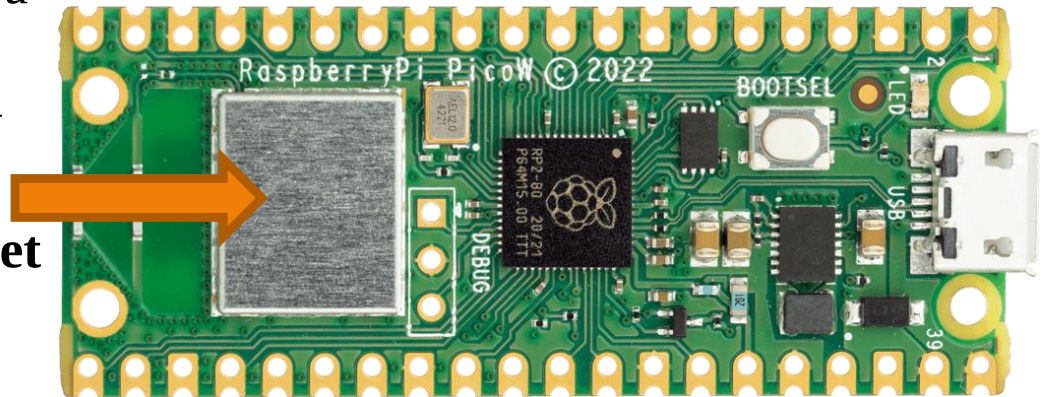
Major Limitation:
No built-in WiFi module
in the Raspberry Pi Pico.

Raspberry Pi Pico W (based on RP2040)

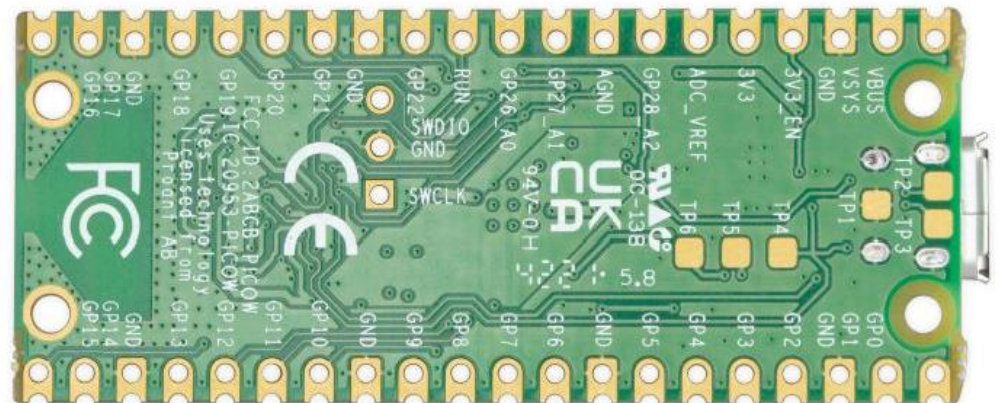
- **Raspberry Pi Pico W** was released in **January 2022** with a retail price of **\$6**.
- A big change in the **Raspberry Pi Pico W** is the adding a **2.4 GHz 802.11b/g/n Wi-Fi chipset** and an on-board **antenna**.
- This is done without changing the **device's form factor (dimensions and pin layout)**, so we can swap a **Pico W** in for a **standard Pico** in any of your existing projects!!!
- The rest of the features and software support are almost unchanged.

[Release news of Raspberry Pi Pico W](#)

Raspberry Pi Pico W (RP2040)
Front side

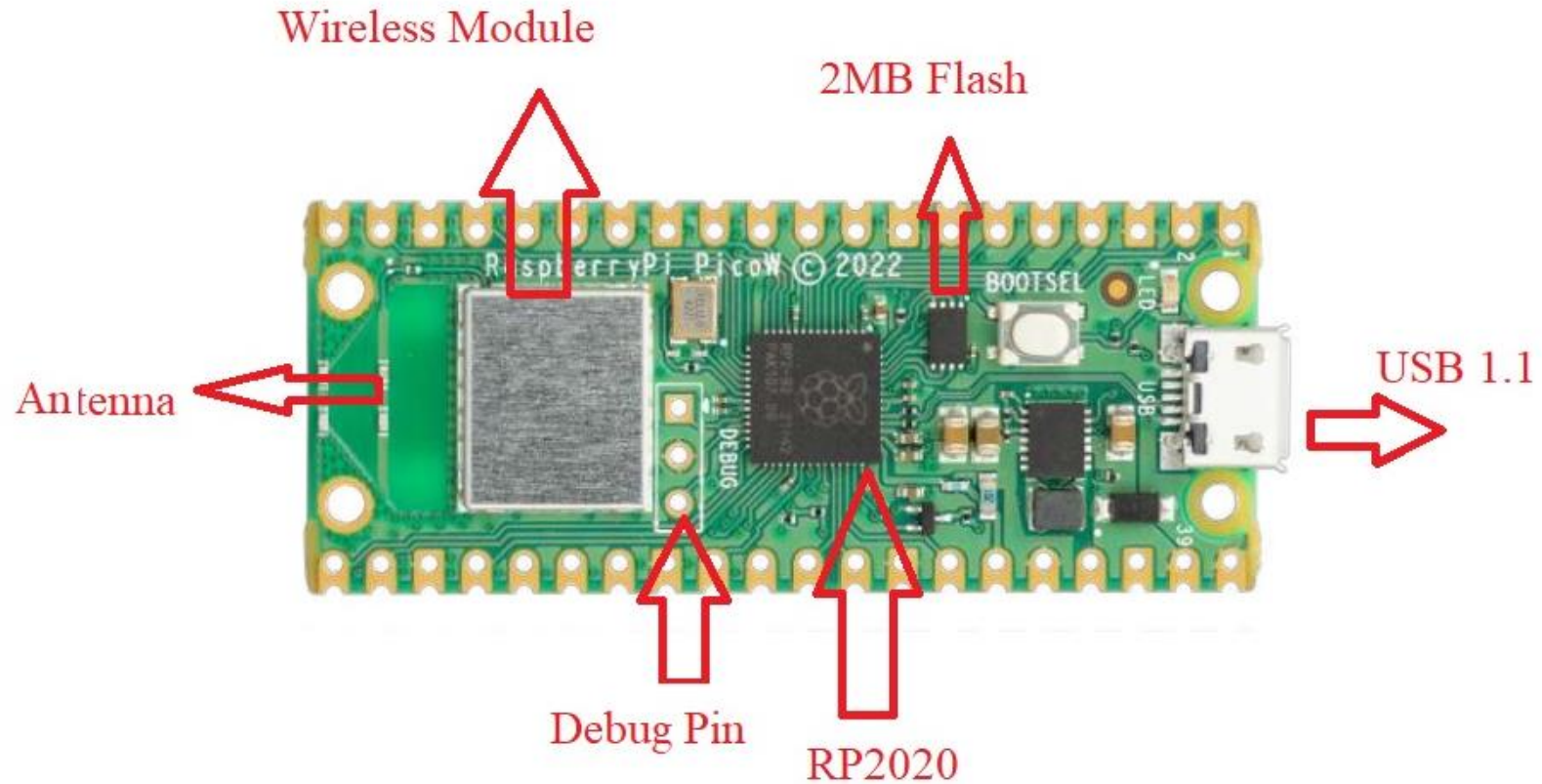


Back side

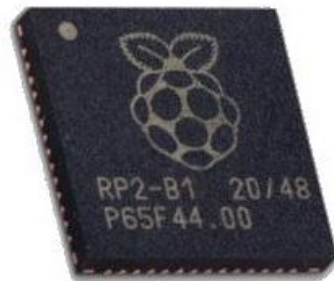


Pin compatible with Raspberry Pi Pico

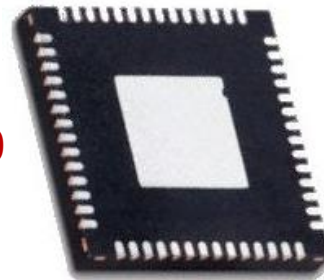
Raspberry Pi Pico W: Layout



RP2040 (Introduction)



RP2040



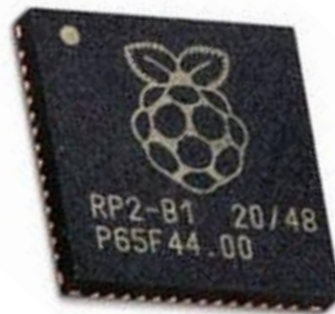
56 pin SMD

SMD: Surface Mounted Device

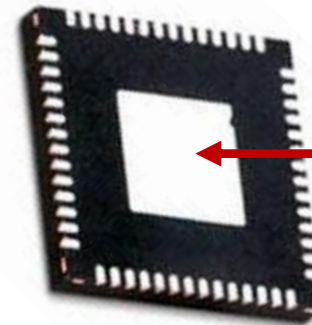
RP2040: Chip



SMD:
Surface
Mounted
Device



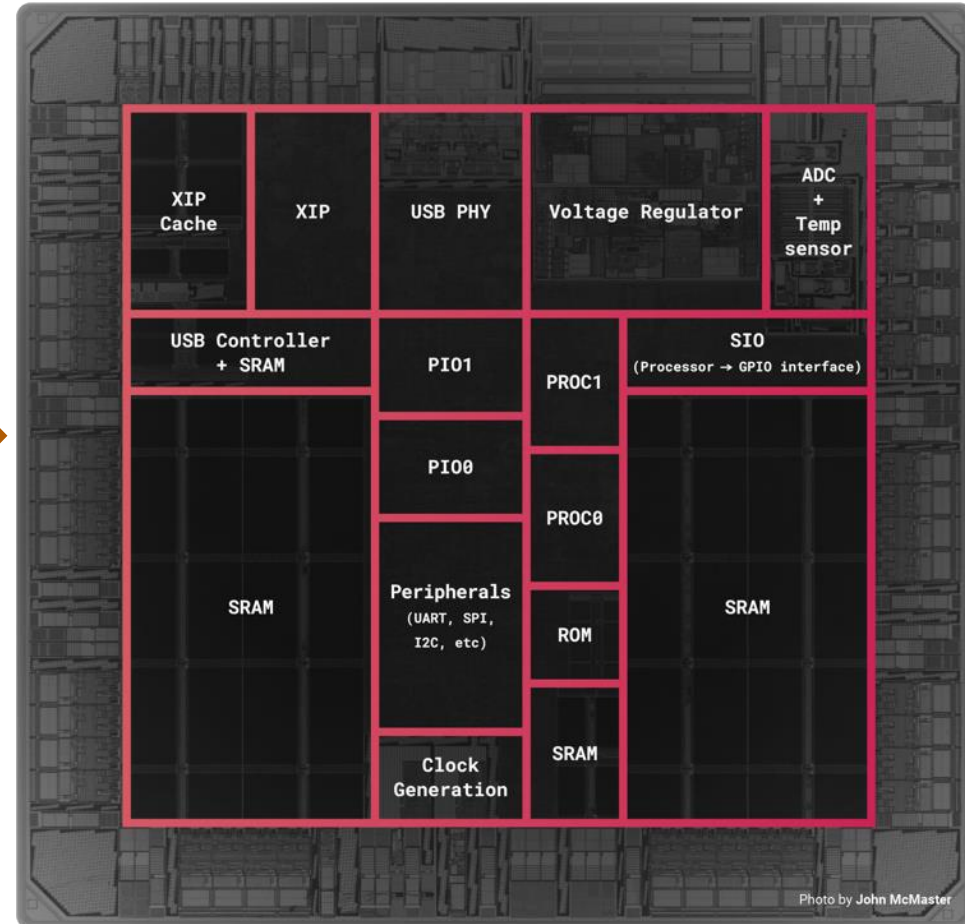
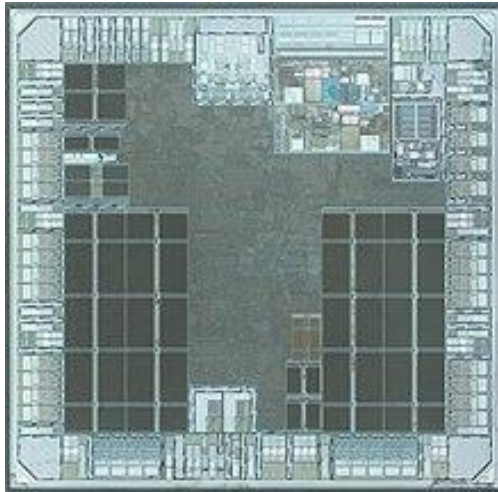
Front side



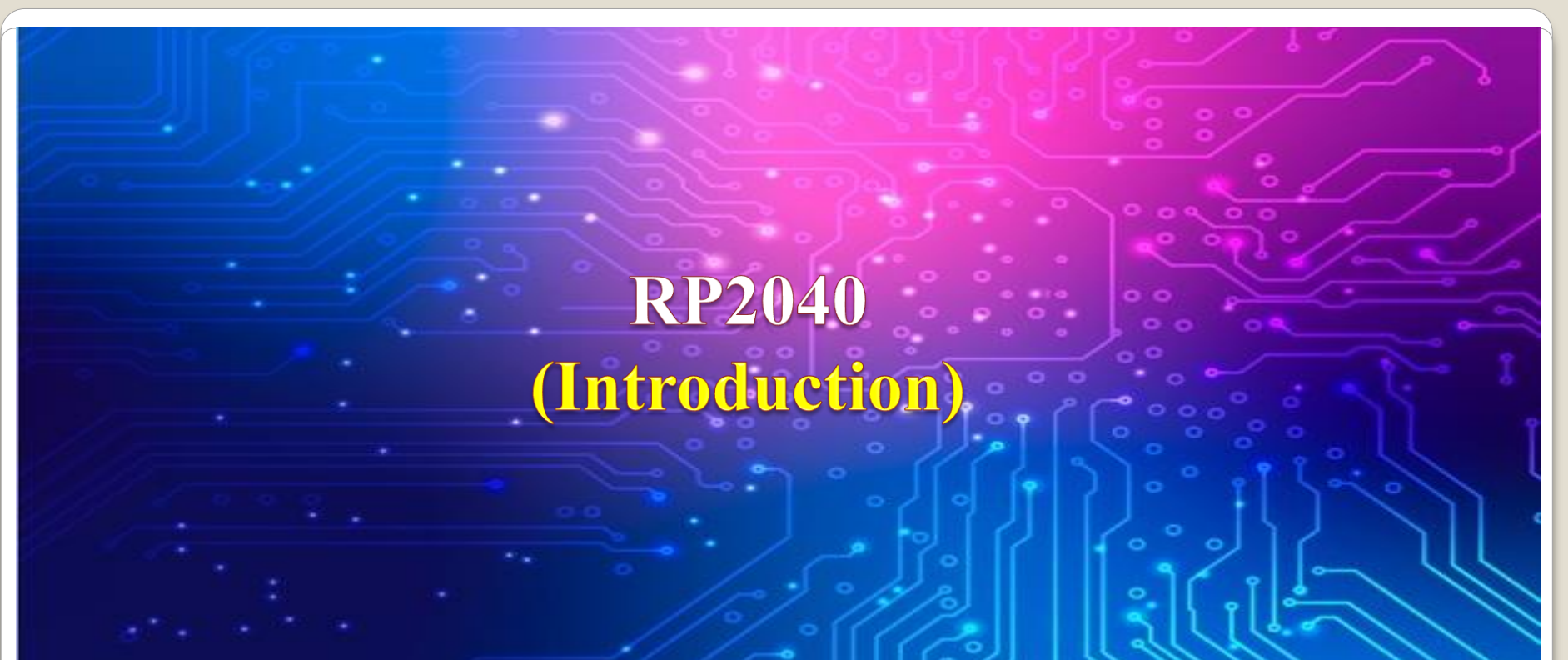
Back side

GND

RP2040: Die Layout



[Ref Link: RP2040 Die](#)

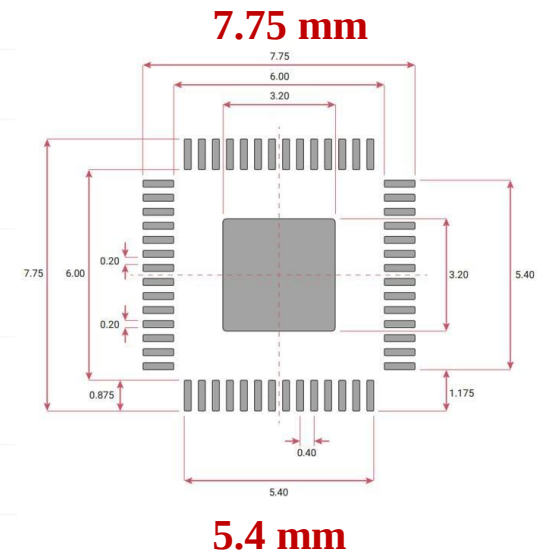
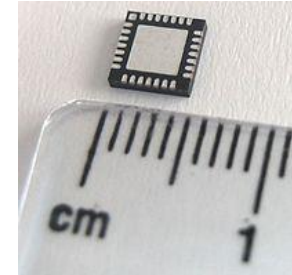


RP2040 (Introduction)

RP2040: Specification

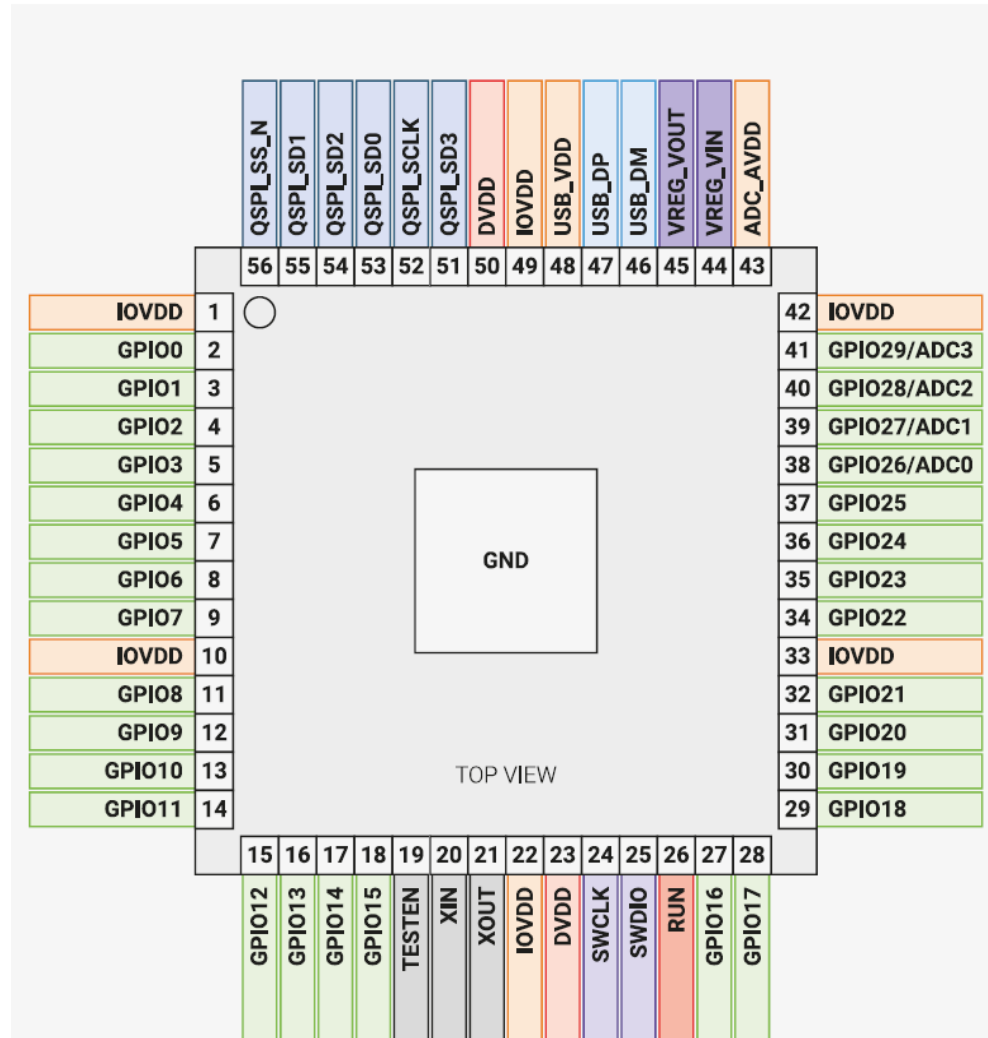
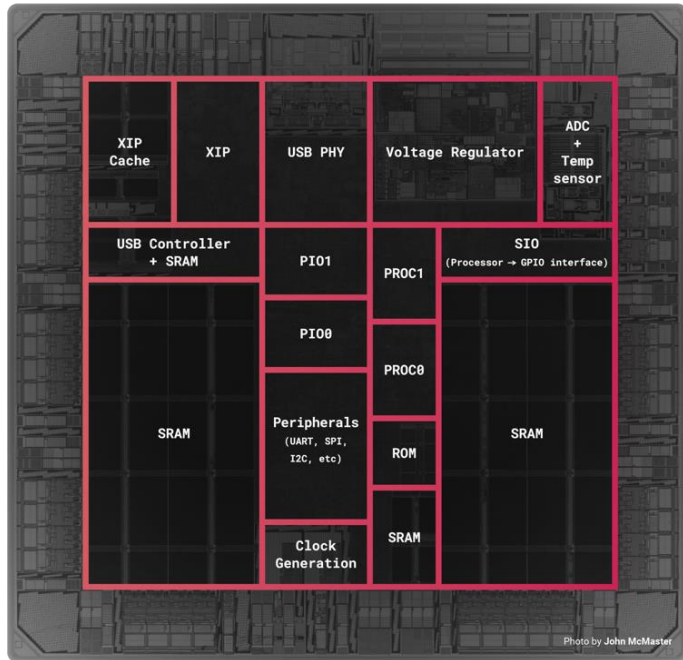
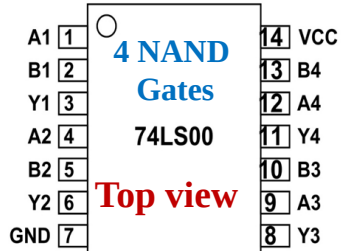
Brand	Raspberry Pi	Family Name	ARM
Package Type	QFN	Mounting Type	Surface Mount
Pin Count	56	Device Core	ARM Cortex M0+
Data Bus Width	32bit	Program Memory Size	16 MB
Maximum Frequency	133MHz	RAM Size	264 kB
USB Channels	1 x Device, 1 x Host	Number of PWM Units	2 x 32 bit
Number of SPI Channels	2	Typical Operating Supply Voltage	3.63 V
Height	0.9mm	Width	7.75mm
Minimum Operating Temperature	-20 °C	Dimensions	7 x 7 x 0.9mm
Instruction Set Architecture	RISC	Number of ADC Units	1
Maximum Operating Temperature	+85 °C	Number of UART Channels	2
Program Memory Type	Flash	Length	7.75mm
ADCs	4 x 12 bit	Number of Timers	4
Number of I2C Channels	2	Country of Origin	TW

QFN: Quad-Flat No-leads

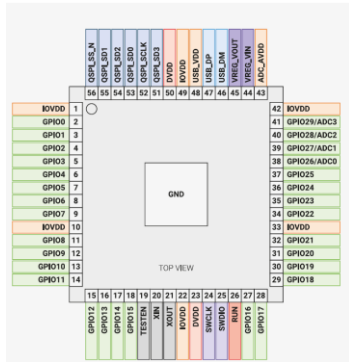


TW: Taiwan

RP2040: Pin Details



RP2040: Pin Details



GPIOx

General-purpose digital input and output

GPIOx/ADCy

General-purpose digital input and output, with analogue-to-digital converter function

QSPIx

Interface to an SPI, Dual-SPI or Quad-SPI Flash device, with execute-in-place support

USB_DM and USB_DP

USB controller, supporting full-speed device and full-/low-speed host

XIN and XOUT

Connect a crystal to RP2040's crystal oscillator

RUN

Global asynchronous reset pin; reset when driven low, run when driven high

SWCLK and SWDIO

Access to the internal Serial Wire Debug multi-drop bus; provides debug access to both processors

TESTEN

Factory test mode pin

GND

Single external ground connection, bonded to a number of internal ground pads on the RP2040 die

IOVDD

Power supply for digital GPIOs, nominal voltage 1.8 V to 3.3 V

USB_VDD

Power supply for internal internal USB full-speed PHY, nominal voltage 3.3 V

ADC_AVDD

Power supply for analogue-to-digital converter, nominal voltage 3.3 V

VREG_VIN

Power input for the internal core voltage regulator, nominal voltage 1.8 V to 3.3 V

VREG_VOUT

Power output for the internal core voltage regulator, nominal voltage 1.1 V, 100 mA max current

DVDD

Digital core power supply, nominal voltage 1.1 V

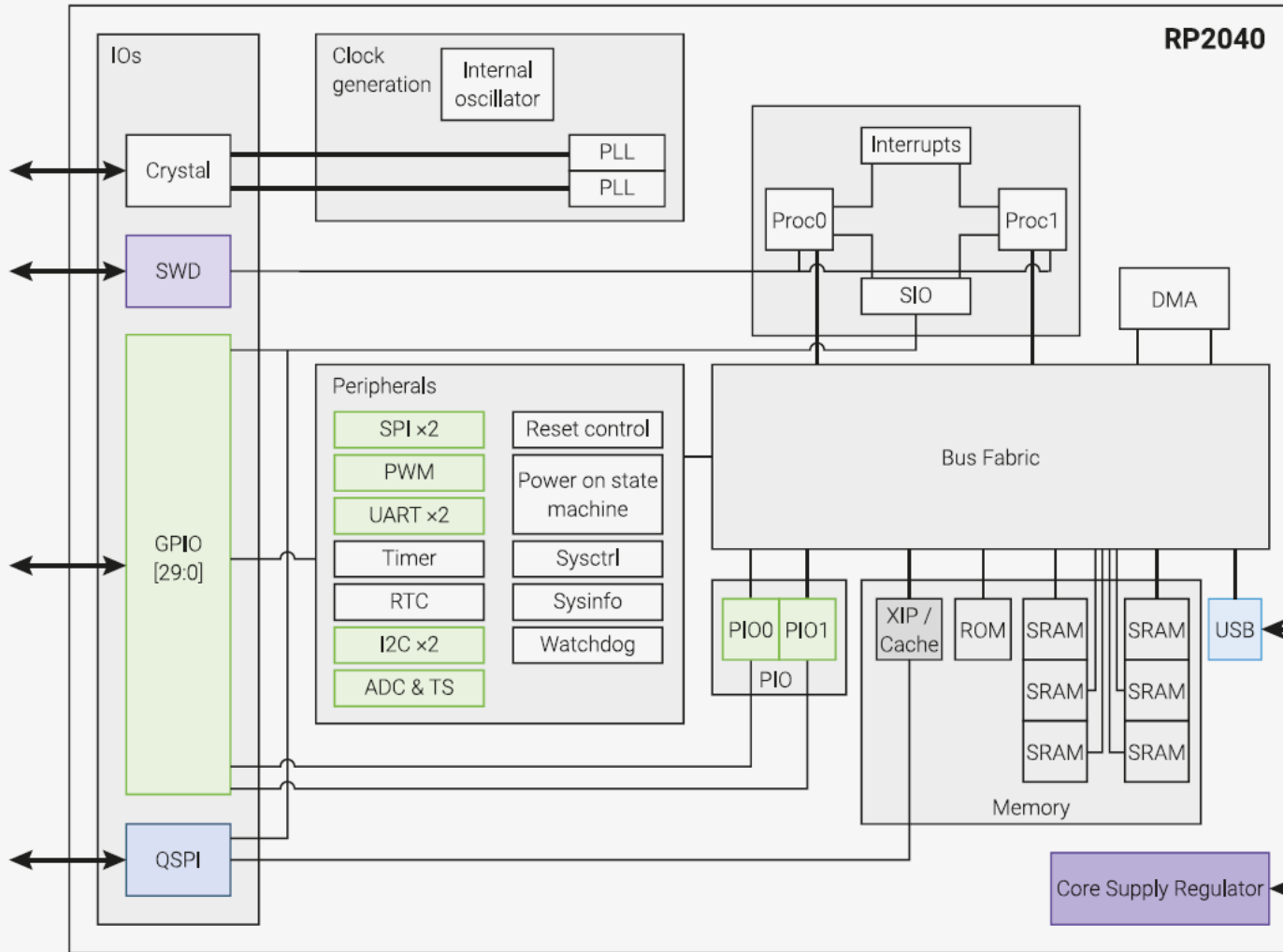
Flash type: NOR Flash

Ref: RP2040 Product Brief



RP2040 (System Overview)

RP2040: System Overview



PLL: Phase Locked Loop

SIO: Single-cycle I/O

SPI: Serial Peripheral Interface

QSPI: Quad SPI

XIP: Execute In Place

SRAM: 264 kB in 6 independent banks

GPIO: General Purpose I/O

UART: Universal Asynchronous Receive/Transmit

PWM: Pulse Width Modulator

RTC: Real-time Clock

I²C: Inter-Integrated Circuit

ADC: Analog-to-Digital Converter

SWD: Serial Wire Debug **PIO:** Programmable I/O **DMA:** Direct Memory Access

RP2040: Key Features



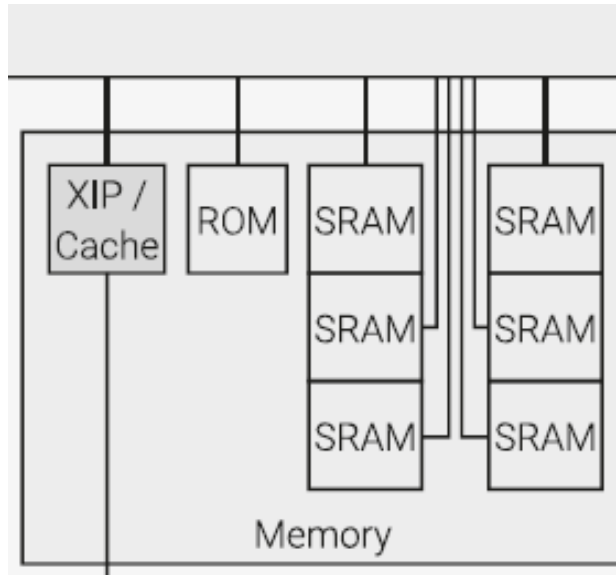
CPU:	Dual ARM Cortex-M0+ @ 133 MHz
Memory:	264kB on-chip SRAM in six independent banks Support for up to 16MB of off-chip Flash memory via dedicated QSPI bus
Architecture:	DMA controller Fully connected AHB crossbar Interpolator and integer divider peripherals On-chip programmable LDO to generate core voltage Two on-chip PLLs to generate USB and core clocks
Interfacing:	30 GPIO pins, four of which can be used as analogue inputs
Peripherals:	2 × UARTs 2 × SPI controllers 2 × I2C controllers 16 × PWM channels 1 × USB 1.1 controller and PHY, with host and device support 8 × PIO state machines
Package:	7 × 7 mm QFN-56 package
Product lifetime:	Raspberry Pi understands the value to customers of long term availability of product and therefore aims to continue supply for as long as practically possible. We expect RP2040 to remain in production until at least January 2041.

LDO:
Linear
Low-Dropout
Regulator

AHB:
Advanced
High Performance
Bus

Ref: RP2040 Product Brief

RP2040: ROM/RAM Details



ROM: 16 kB

Internal SRAM: 264 kB

There are 6 banks: 256 kB + 8 kB

4 banks: 64 kB each = 256 kB

2 banks: 4 kB each = 8 kB

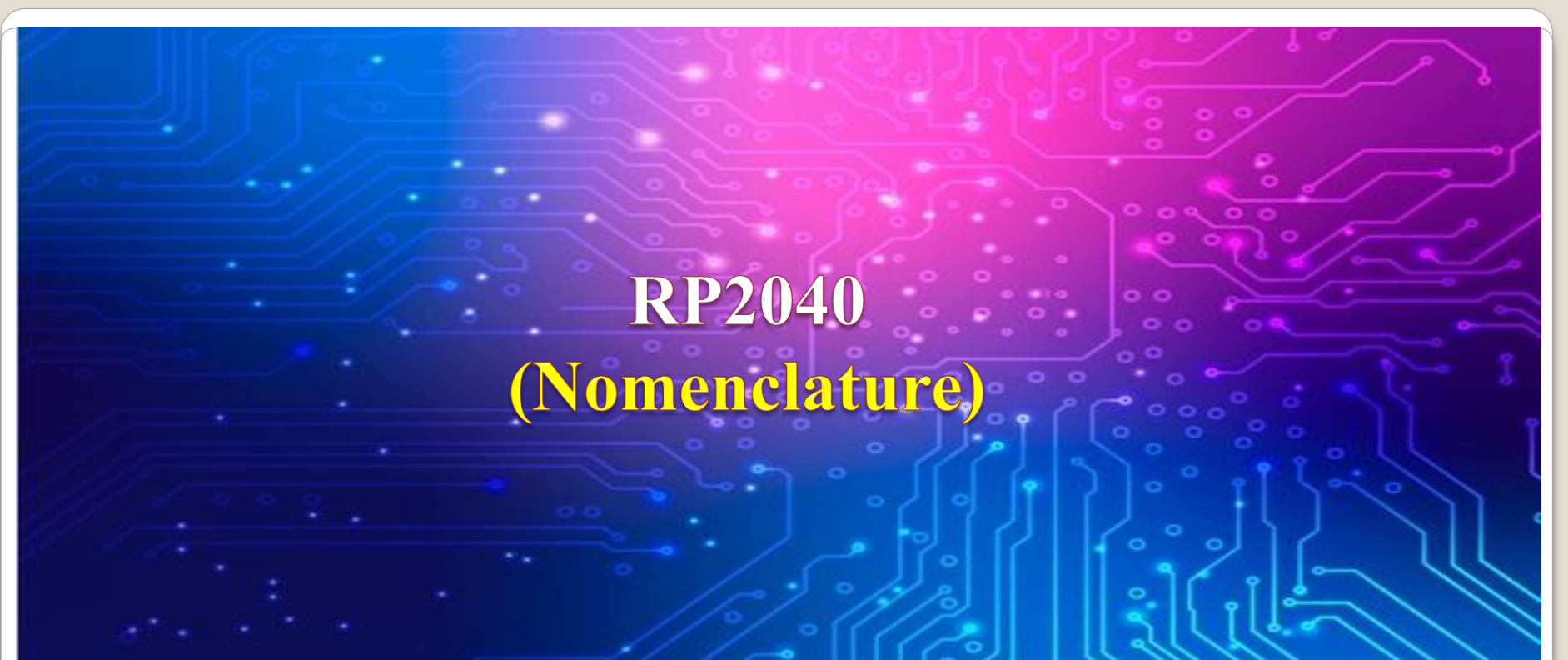
Additional RAM

XIP/Cache: 16 kB

If USB is not used: **4 kB**

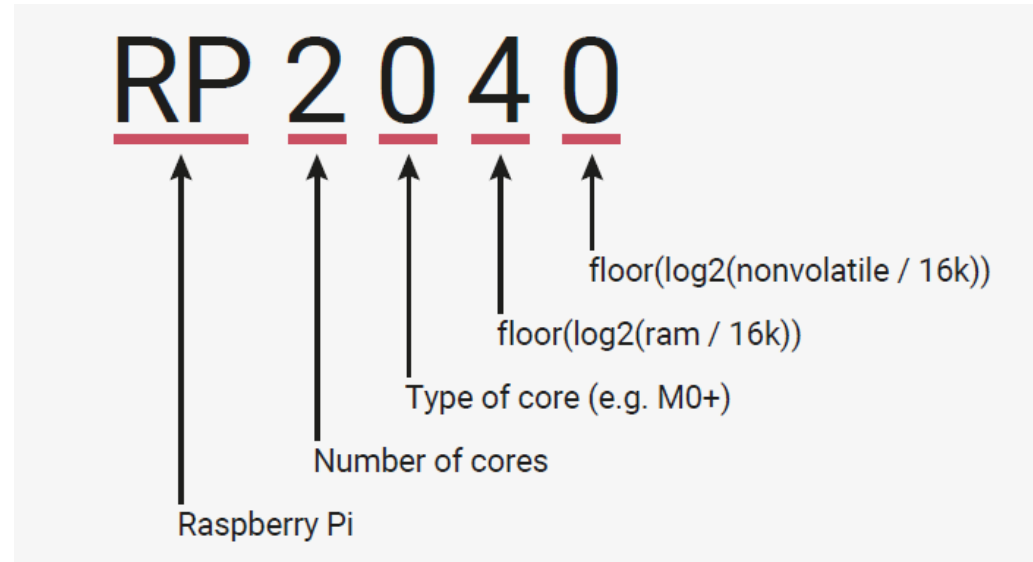
Thus, Max total RAM: $264 + 16 + 4 = 284$ kB

XIP: Execute In Place



RP2040 (Nomenclature)

RP2040: Nomenclature

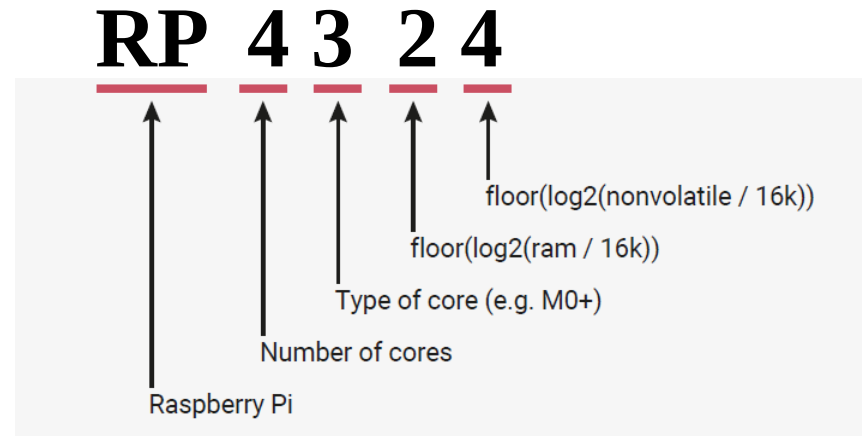


- Number of processor cores (**2**)
- Type of processor (M**0**+)
- $\text{floor}(\log_2(\text{ram} / 16\text{k})) \Rightarrow \text{SRAM} = 264\text{k} \Rightarrow \log_2(264\text{k} / 16\text{k}) \Rightarrow \text{floor}(\log_2(16.5)) \Rightarrow \mathbf{4}$
- $\text{floor}(\log_2(\text{nonvolatile} / 16\text{k}))$ or **0** if no onboard nonvolatile storage

Note: 16MB of flash is external to RP2040 and not part of the SoC

Quiz 1: Nomenclature

Q. Find the configuration of this chip if the name is given as here.

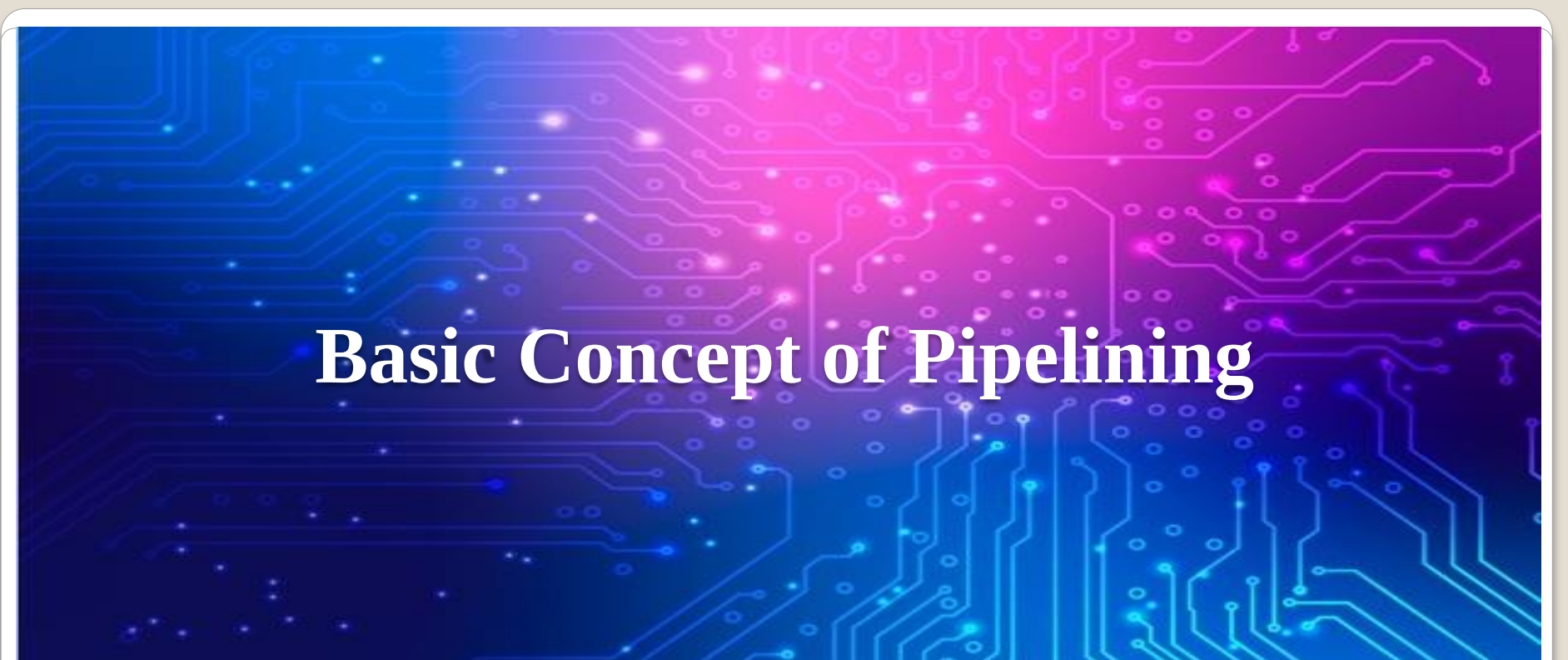


ANS:

- Number of processor cores (**4**)
- Type of processor (M**3**) : **Cortex M3**
- $\text{floor}(\log_2(\text{ram} / 16\text{k})) \Rightarrow 2 \Rightarrow \text{floor}(\log_2(2^2)) \Rightarrow (\text{RAM size} / 16\text{k}) \rightarrow 4$
 $\Rightarrow \text{RAM size} \rightarrow 4 * 16\text{k} \rightarrow \textbf{64k}$
- $\text{floor}(\log_2(\text{nonvolatile} / 16\text{k})) \rightarrow 4 \Rightarrow \text{floor}(\log_2(2^4)) \Rightarrow (\text{Flash size} / 16\text{k}) \rightarrow 16$
 $\Rightarrow \text{Built-in Flash Size} \rightarrow 16 * 16\text{k} \rightarrow \textbf{256k}$

RP2040: Feature Summary

- Dual-core Arm Cortex-M0+ @ 133MHz
- 264KB of on-chip RAM
- Support for up to 16MB of off-chip Flash memory via dedicated QSPI bus
- DMA controller
- 30 GPIO pins, 4 of which can be used as analogue inputs
- 2 × UARTs, 2 × SPI controllers, and 2 × I²C controllers
- 8 × Raspberry Pi Programmable I/O (PIO) state machines
- Accurate clock and timer on-chip
- Temperature sensor
- 16 × PWM channels
- 1 × USB 1.1 controller and PHY, with host and device support
- Accelerated floating-point libraries on-chip
- USB mass-storage boot mode with UF2 support, for drag-and-drop programming



Basic Concept of Pipelining

Introduction to pipelining

Pipelining is a technique of decomposing a sequential process into sub-operations, with each sub-process being executed in a special dedicated segment that operates concurrently with all other segments.

- **Conventional Sequential Execution**

Clock Cycle	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
Instruction 1	IF	ID	EX	MA	WB										
Instruction 2						IF	ID	EX	MA	WB					
Instruction 3											IF	ID	EX	MA	WB

IF-> Instruction Fetch

ID-> Instruction Decode

EX-> Execute

MA-> Memory Access

WB-> Write Back

Introduction to pipelining

- **Pipelined Execution**

Clock Cycle	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
Instruction 1	IF	ID	EX	MA	WB										
Instruction 2		IF	ID	EX	MA	WB									
Instruction 3			IF	ID	EX	MA	WB								

- **Memory Access Is Only Required for Load/Store**

- RISC architectures follow a **load/store** design, meaning only load (LW) and store (SW) instructions access memory.
- Arithmetic and logical instructions operate on registers, not memory.

Basics of Pipelining

- **Fetch:**
 - Retrieves the next instruction from memory.
 - Handles sequential instructions or computes the address for branches/jumps.
- **Decode:**
 - Breaks down the instruction to determine:
 - Type of operation (e.g., addition, logical shift).
 - Registers to read/write.
 - Additional control information.
- **Execute:**
 - Uses the ALU to perform computations.
 - Executes the operation specified by the Decode stage.
- **Memory Access:**
 - Reads data from or writes data to memory, if required.
- **Write Back:**
 - Writes the results of the instruction back into registers.

Basics of Pipelining

- **Pipelining in Processor Design:**
- **Pipeline Stages:**
 - Simplifies logic by dividing it into stages.
 - Example stages: Fetch, Decode, Execute, Memory Access, Write Back.
- **Pipeline Depth Examples:**
 - Simple processors: 3–6 stages (e.g., Arm Cortex-M3, M7).
 - Complex processors: 8–14 stages (e.g., Arm Cortex-A7, Intel i3/i5/i7).
 - Extreme example: Pentium 4 Cedar Mill with 31 stages.
- **Trade-Offs in Pipelining:**
 - **Advantage:** Increases clock speed by splitting logic into more stages.
 - **Disadvantage:** Too many stages can introduce overhead and inefficiency (e.g., branch misprediction penalties).
 - Balance between performance and complexity is essential.
- **Pipeline Stage Variations:**
 - Heavy stages (e.g., Execute) can be subdivided further for better performance.

Basics of Pipelining

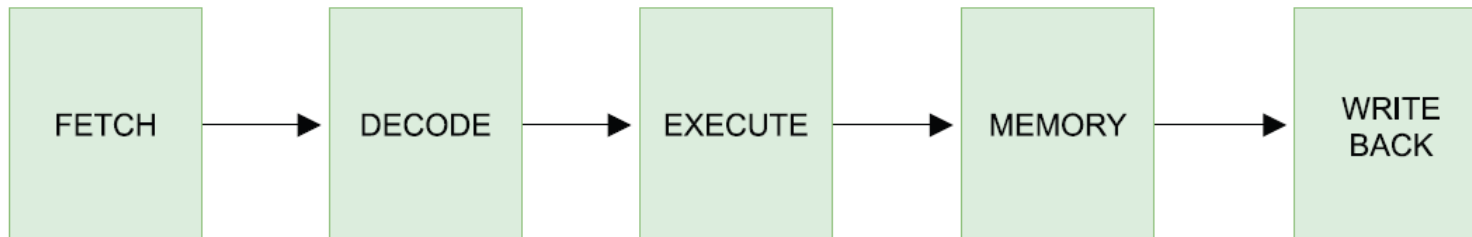


Figure 2.6 Logic blocks of a typical processor

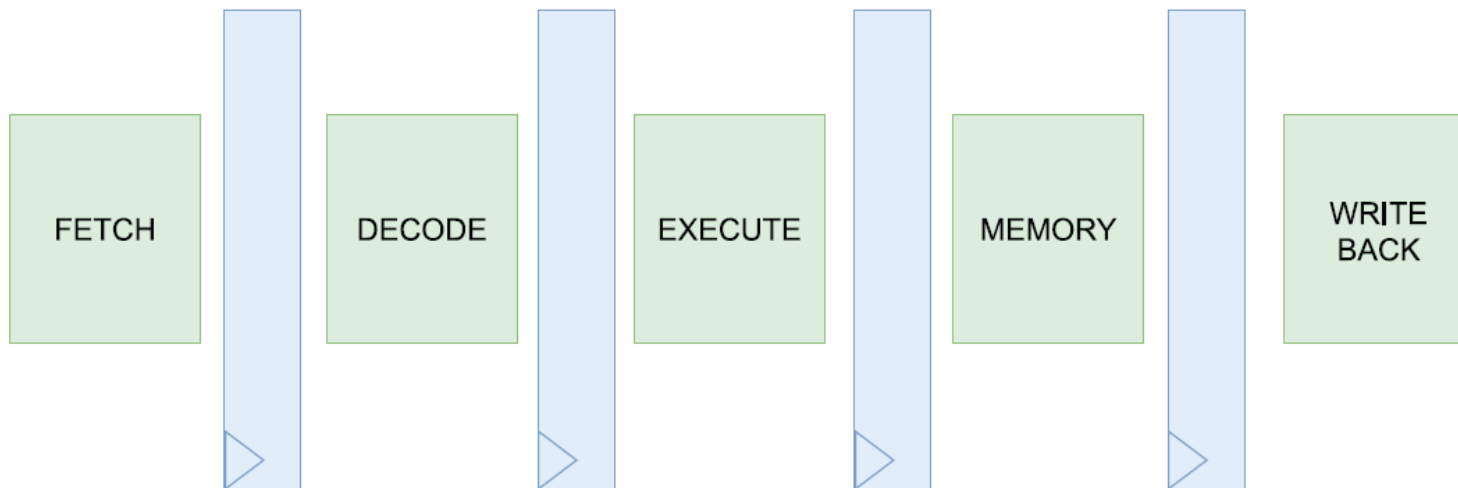
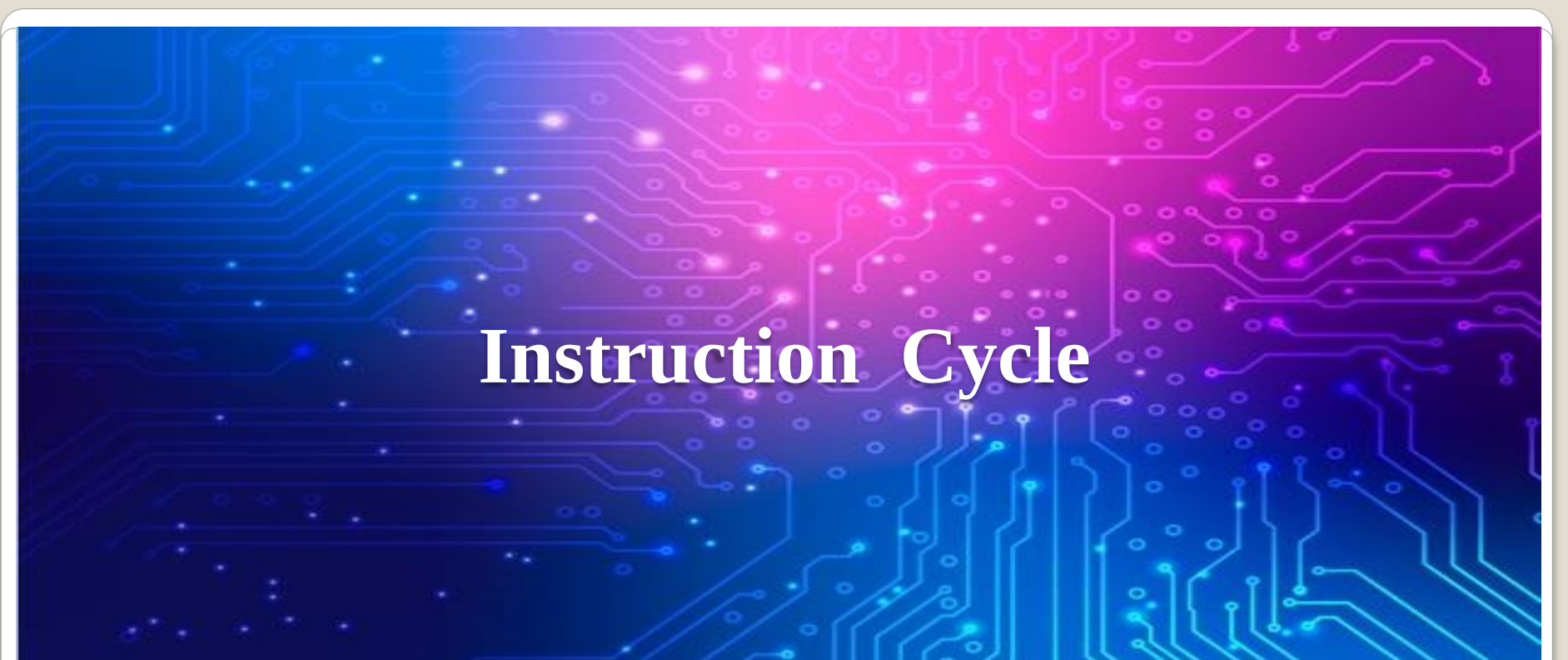


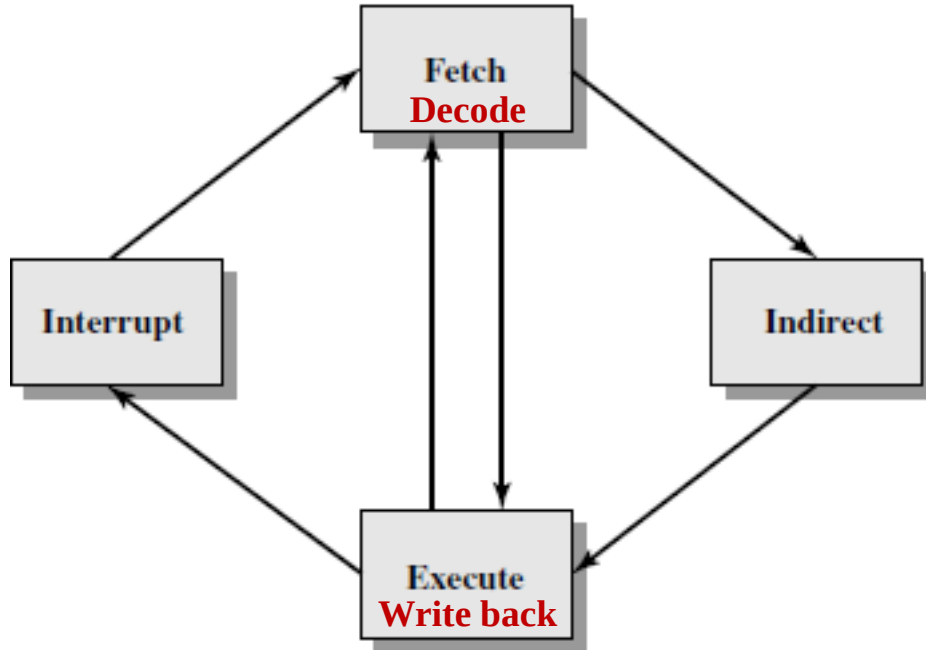
Figure 2.7 Five-stage RISC processor pipeline

Activate Windows
Go to Settings to activate



Instruction Cycle

Instruction Cycle



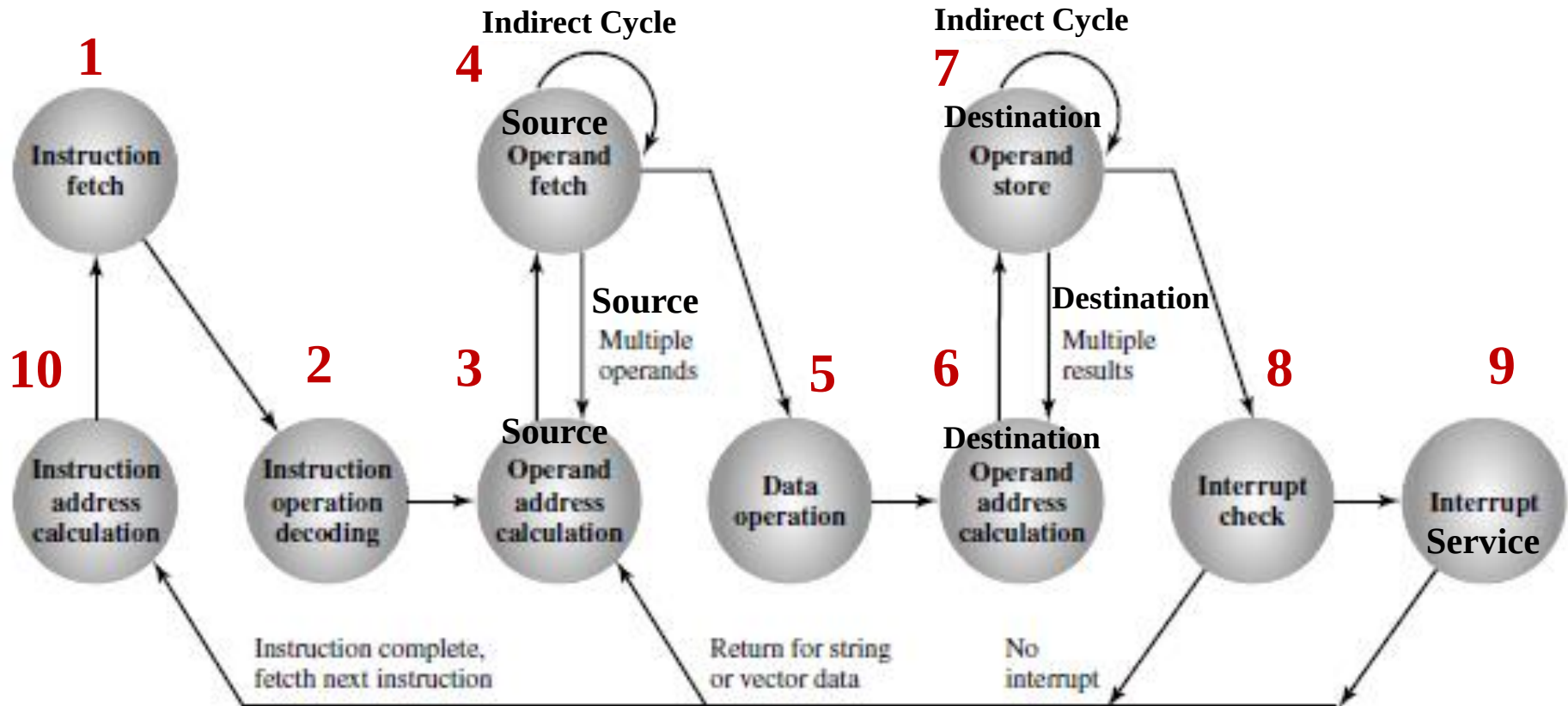
- **Interrupt:** After the completion of execution of every instruction, if interrupts are enabled and an interrupt has occurred, save the current CPU state and service the interrupt.

- **Fetch:** Read the next instruction from the memory into the processor.
- **Decode:** By decoding the opcode, the CPU understands the operation to be performed.
- **Indirect:** If the operands are from memory, initiate memory Read cycle (**indirect**), if not, read the operands from the registers (**direct**)
- **Execute:** Perform the indicated operation on the operands.
- **Write back:** Write the results back into a register.

ADD R1, R2, R3 **Direct addressing**
 $R1 \leftarrow R2 + R3$

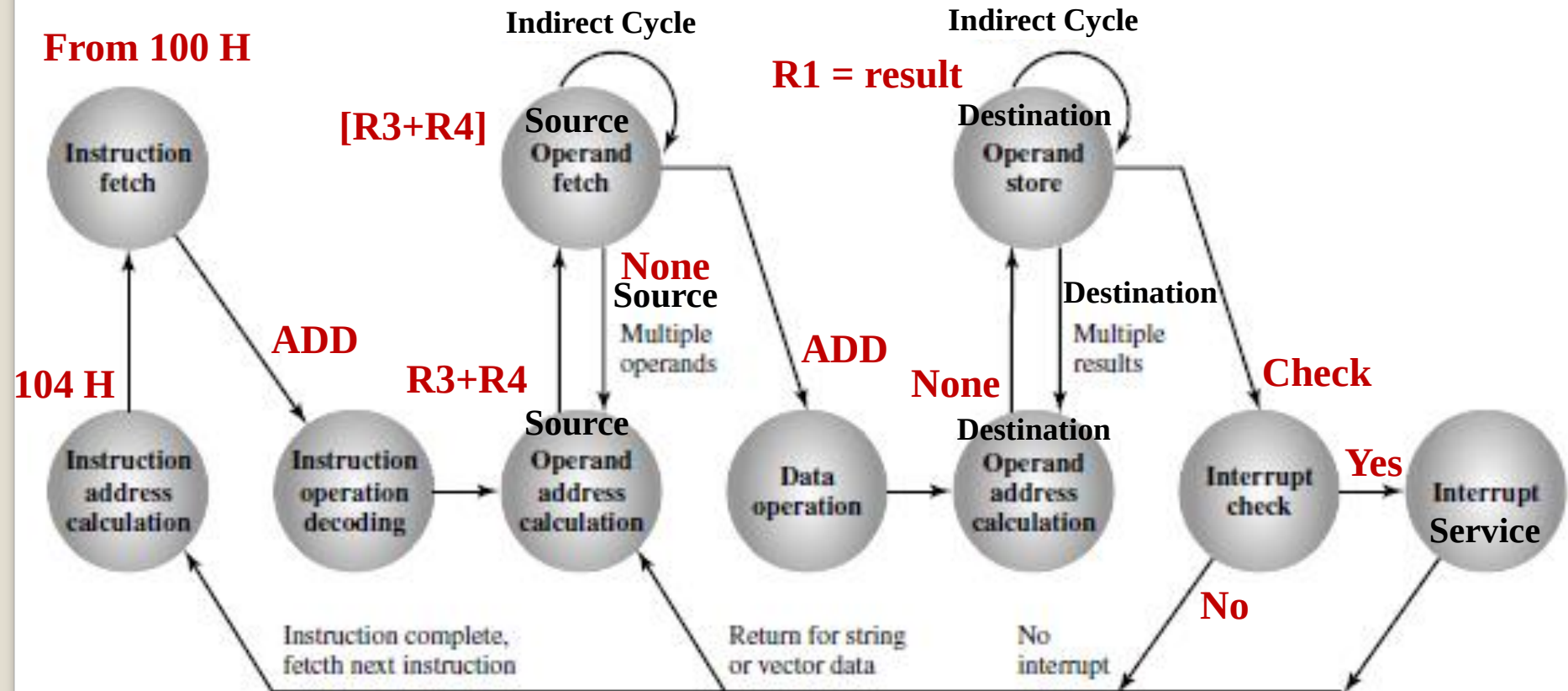
ADD R1, R2, [R3] **Indirect addressing**
 $R1 \leftarrow R2 + [R3]$

Instruction Cycle: State Transition Diagram



Let us understand this better with an instruction next ...

Instruction Cycle: ADD Instruction



PC → 100H ADD R1, R2, [R3+R4]

New PC → 104H

Note: Normally, ALU is not used to compute the address of the next instruction, a dedicated adder takes care of it.

Instruction format: 32 bits wide (4 bytes)

opcode	dest	src1	src2
--------	------	------	------

**Operation
to be
performed**

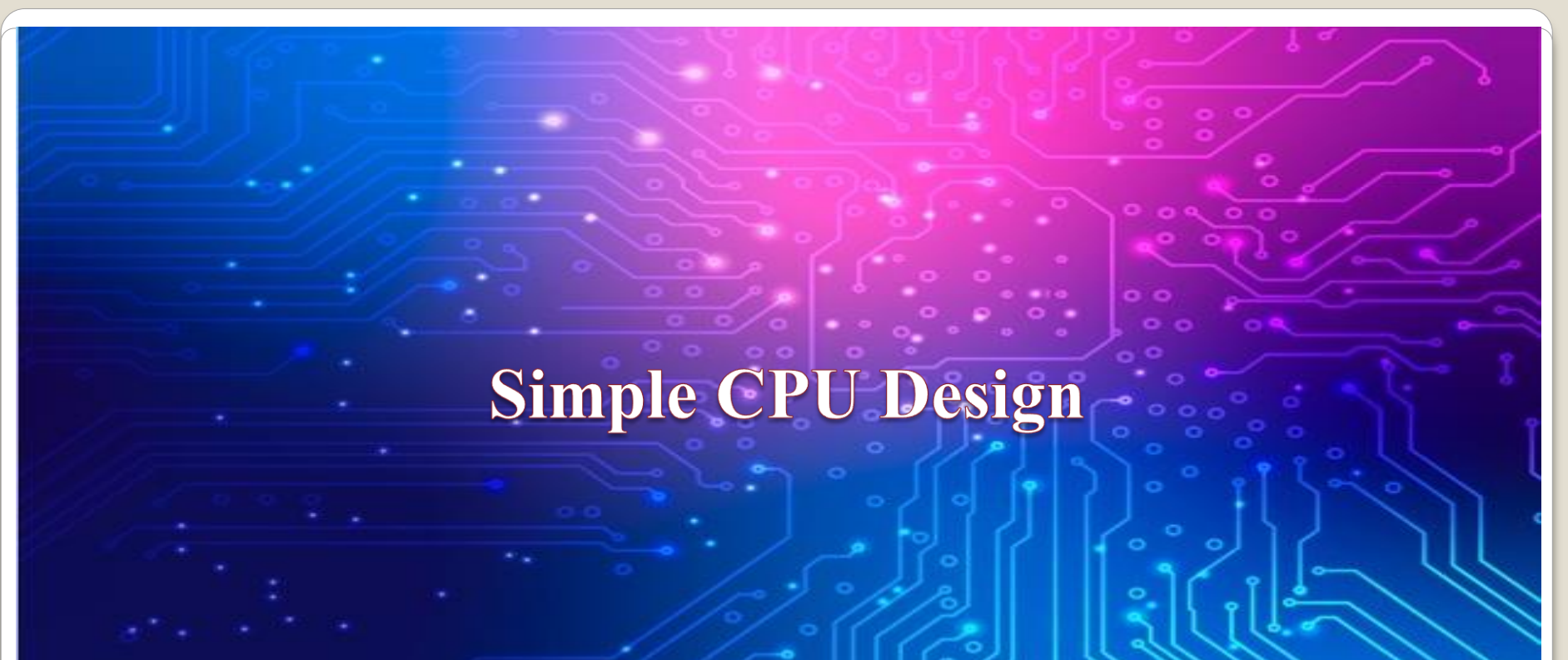
$$R1 \leftarrow R2 + [R3+R4]$$

Instruction Execution: Unpipelined

(Many operations may take different CPU cycles based on the type of instructions)



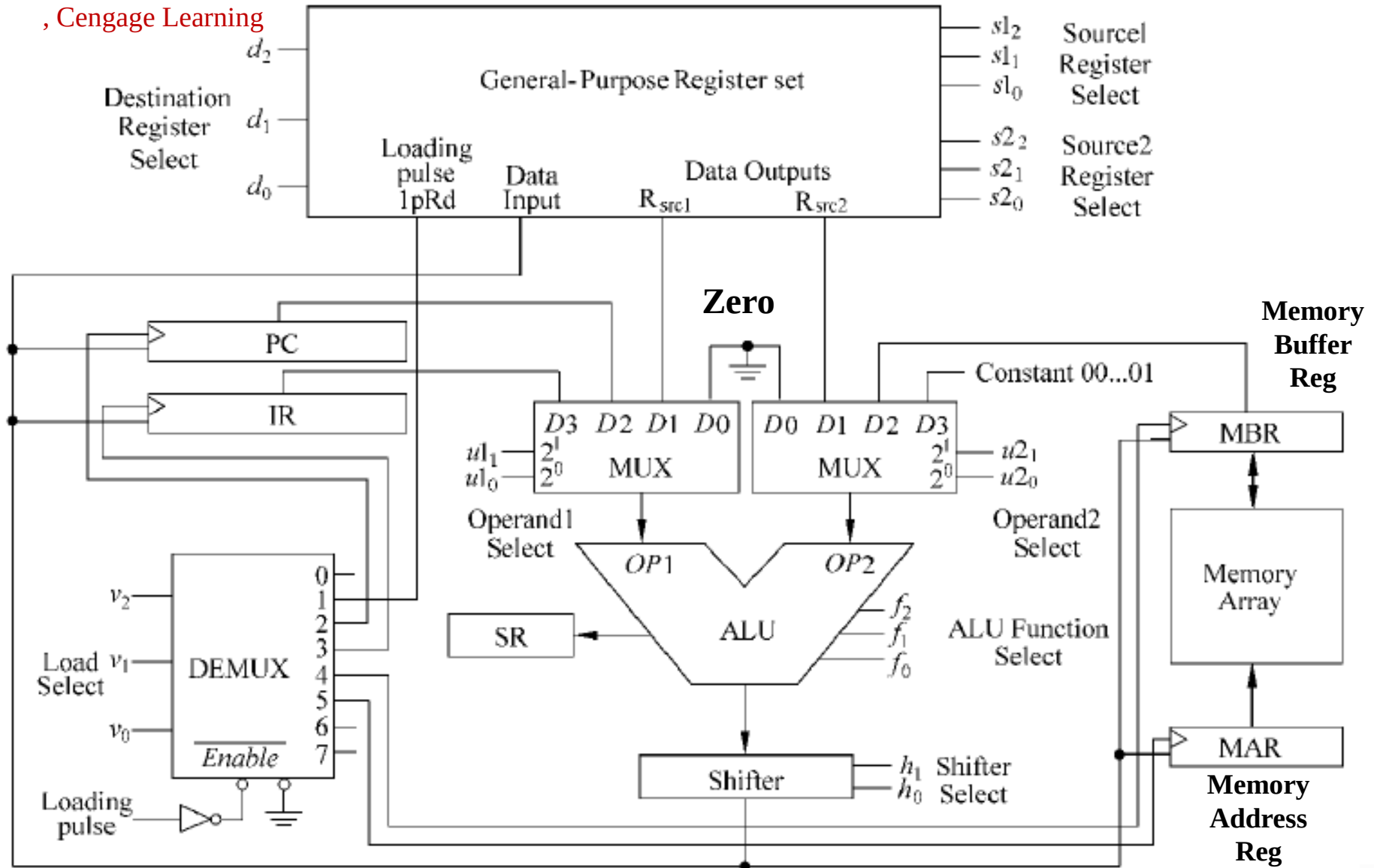
- Instruction **Fetch**: Instruction is read from the program memory.
- **Decode**: Instruction is decoded to know the operation to be performed by the CPU.
- Depending upon the type of instruction the following operations are done.
 - **Operand fetch** (no operand or single operand or multiple operands).
 - Types of the operand fetch could either be direct or indirect .
 - **Direct**: One of the registers itself is an operand.
 - **Indirect**: Operand is in memory. Compute the address based on the addressing mode.
 - Then, an Indirect cycle is started to get the operand from the memory.
 - Operand is brought into the CPU's internal general purpose register.
- **Execute**: Operation is performed by the ALU.
- **Write back**: Results are stored back into a register or memory if the instruction is a store.
- All these operations are done over a few cycles before the next instruction is brought into the CPU. – **Unpipelined execution.**



Simple CPU Design

CPU Design: General Purpose Reg Based

Ref: Computer Organization-Principles, Analysis, and Design –By Lan Jin and Bo Hatfield, Cengage Learning



CPU Design: General Purpose Reg Based



Ref: Computer Organization-Principles, Analysis, and Design –By Lan Jin and Bo Hatfield,
Cengage Learning

➤ **Program Counter (PC) :**

It contains the address of an instruction to be executed next. The PC is updated by the CPU after each instruction is executed so that it always points to the next instruction to be executed. A branch or skip instruction will also modify the content of the PC.

➤ **Instruction Register (IR) :**

It contains the instruction most recently fetched or executed. The fetched instruction is loaded into an IR, where the opcode and operand specifier is analyzed.

➤ **Memory Buffer (or Data) Register (MBR or MDR) :**

It contains a word of data to be written to memory or the words most recently read. Contents of MBR are directly connected to the data bus.

➤ **General Purpose Registers:**

These components are general use memory storage in the CPU that can be accessed very fast (as compared to RAM); they are created from combining latches with a decoder. The latches create circuitry that can remember while the decoder creates a way for individual memory locations to be selected.

CPU Design: General Purpose Reg Based



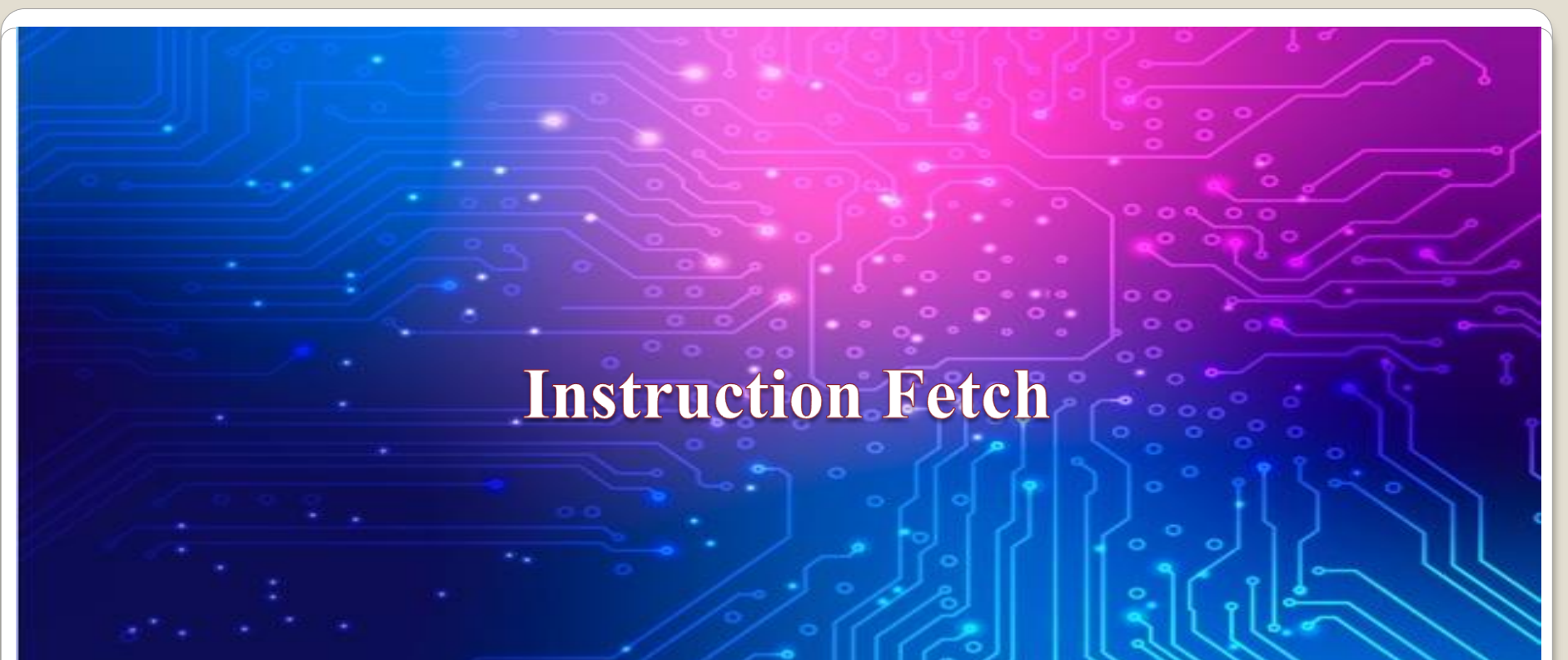
Ref: Computer Organization-Principles, Analysis, and Design –By Lan Jin and Bo Hatfield,

Cengage Learning

MAR:

The functioning of a Memory Address Register can be explained according to the following steps:

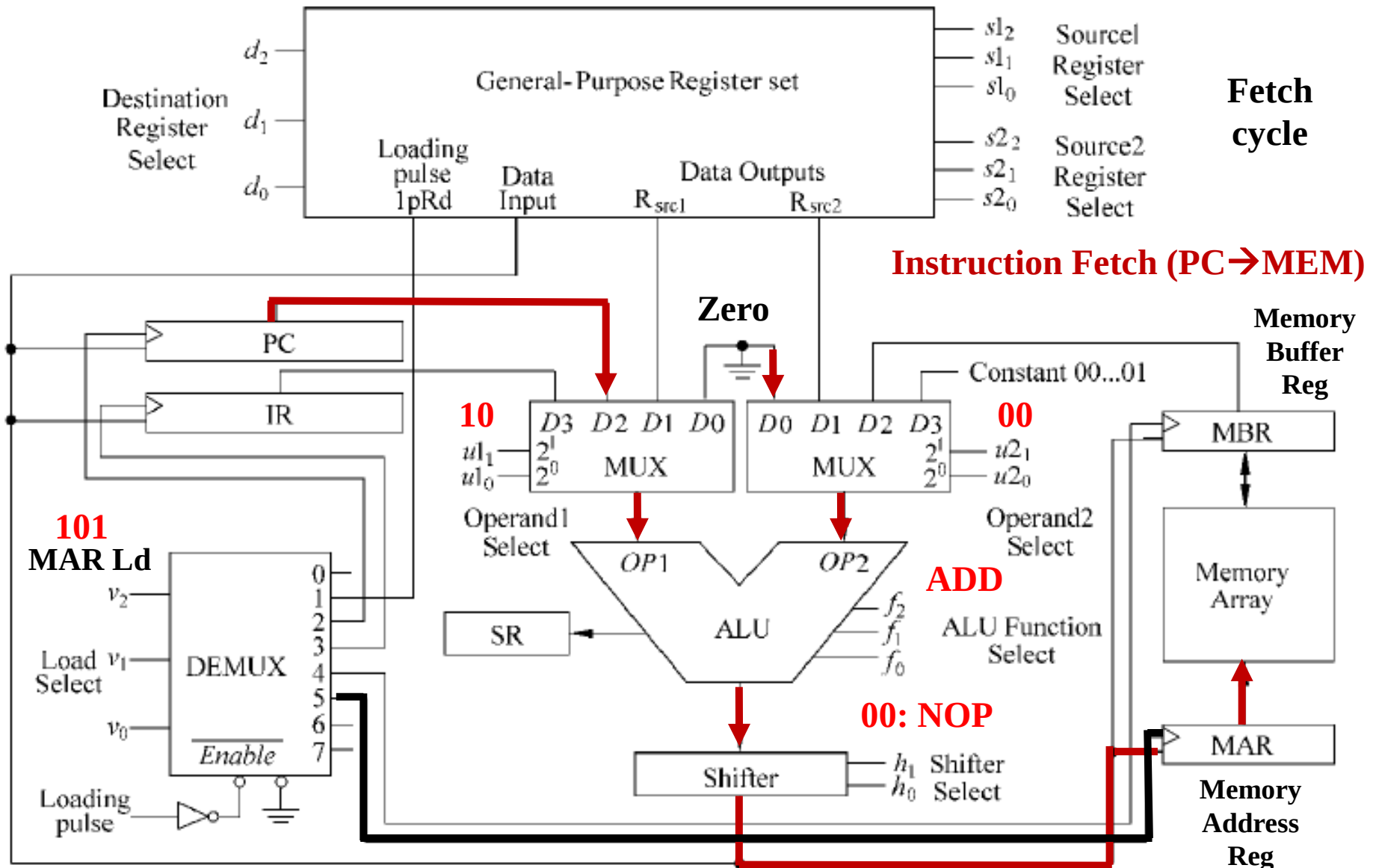
- It holds the memory location of the next piece of data or instruction that will be processed.
- The processor fetches this memory location from the MAR.
- The data or instruction at the fetched address is then read or written.
- Example: Consider the scenario of a spreadsheet application. The user selects a cell to update the content. As part of this process, the processor fetches the instruction that modifies the cell content. The MAR holds the address of this instruction in memory, which the processor then reads to execute.
- **Difference Between Program Counter and Memory Address Register**
- The **Program Counter (PC)** is a type of register that holds the memory address of the next instruction to be executed by the Central Processing Unit (CPU). Following each instruction fetch, the Program Counter increments, keeping track of the sequential execution while processing software.
- The **Memory Address Register (MAR)**, is more generalised in scope. It holds the memory addresses of data and instructions that a CPU needs to access for its next processing steps. Virtually any read or write cycles within memory can involve the MAR, making it a highly versatile component.



Instruction Fetch

Instruction Fetch: PC → Program mem

Fetch cycle



Instruction Fetch: PC → Program mem



Steps Involved in Fetch Cycle

Fetch Cycle: The Program Counter contains the address of the instruction that has to be next executed at the start of the fetch cycle (PC).

Step 1: The address in the program counter is transferred to the memory address register (MAR), which is the only register connected to the address lines of the system bus.

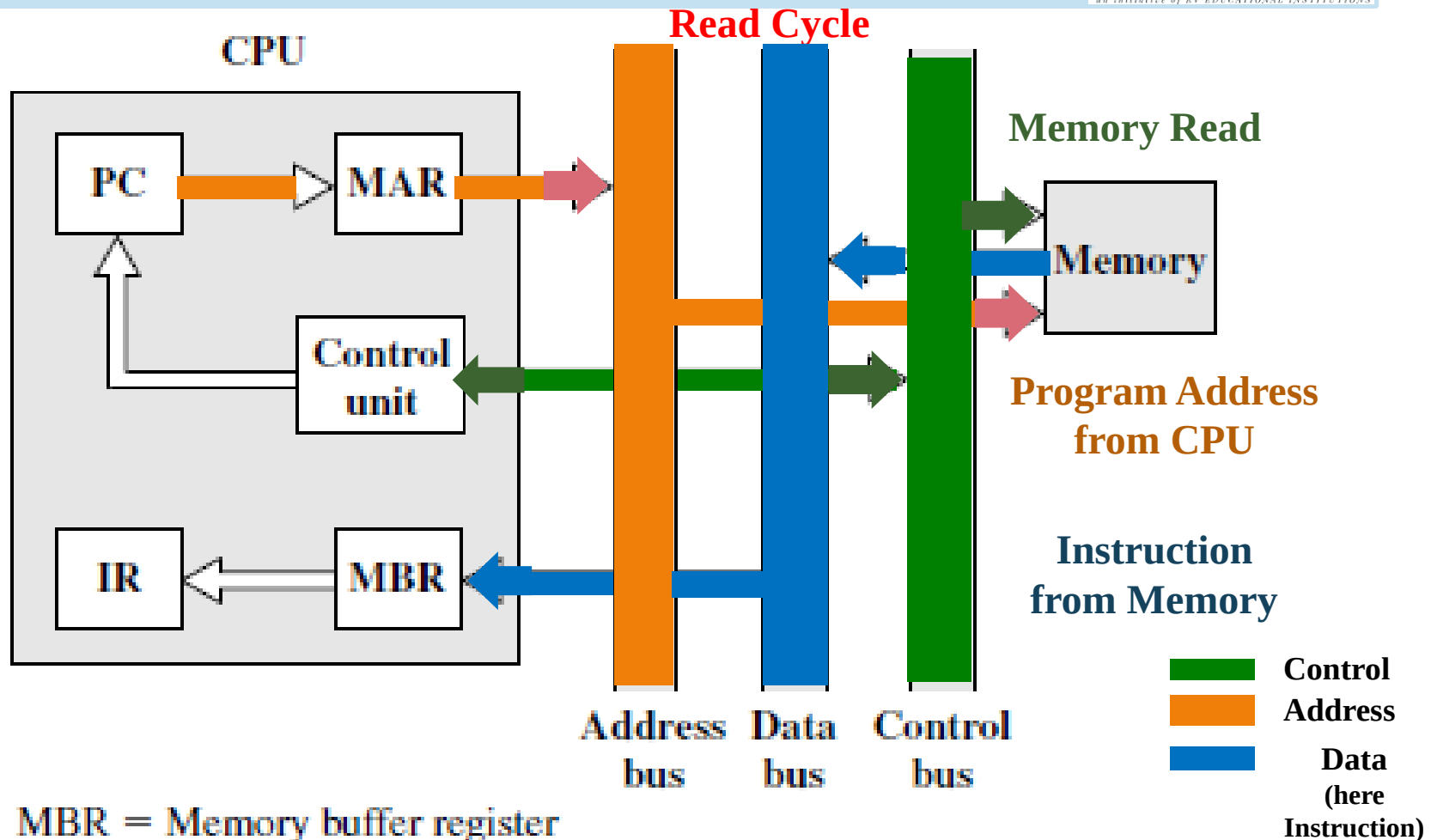
Step 2: The MAR address is added to the address bus. The control unit then issues a READ command on the control bus, and the result is displayed on the data bus before being copied into the memory buffer register (MBR). The program counter is increased by one to prepare for the next instruction. (These two actions can be performed concurrently to save time.)

Step 3: The MBR's content is transferred to the instruction register (IR).

- Move the contents of the PC to MAR.
- MAR-specified memory location contents are to be copied to the MBR. The process increases the PC's content.
- Copy the contents of the MBR to the IR.

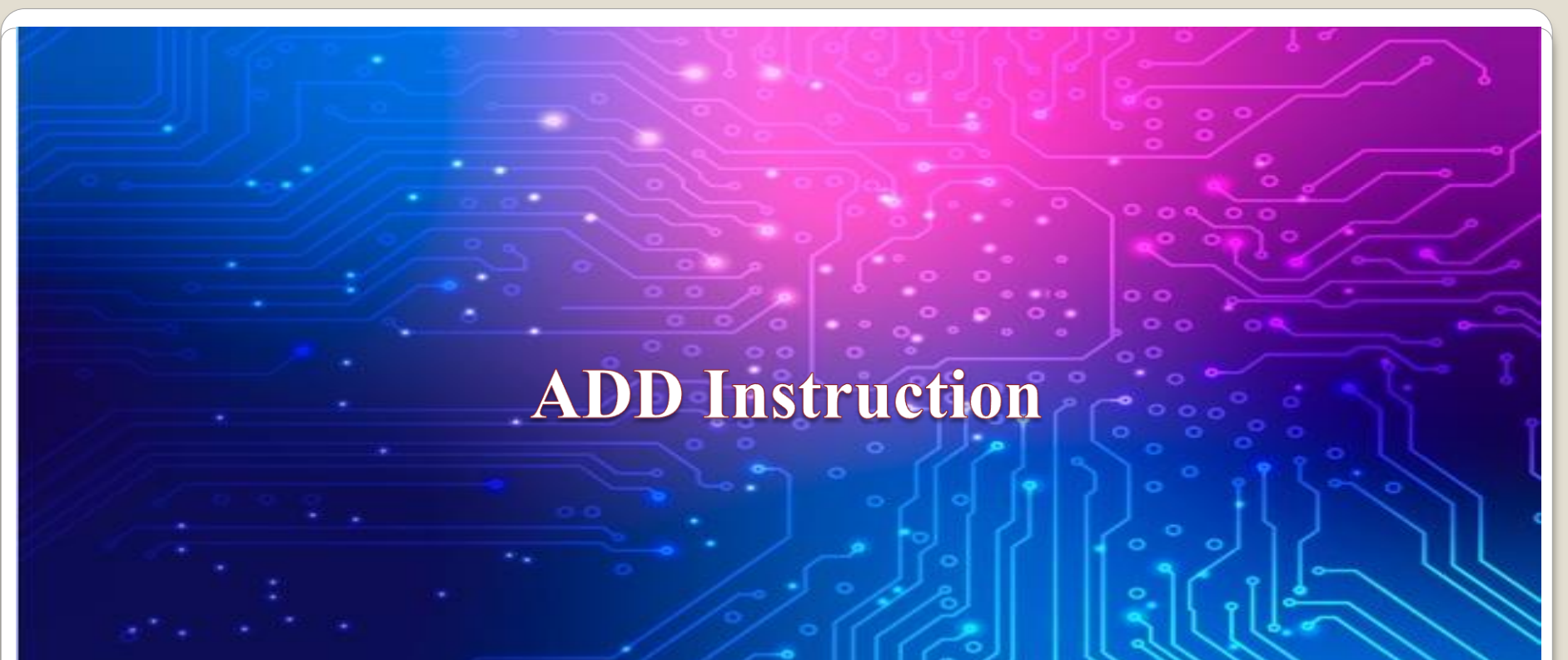
Instruction Fetch – Program Mem Read

(CPU-Memory Interactions explained)



MBR = Memory buffer register
MAR = Memory address register
IR = Instruction register
PC = Program counter

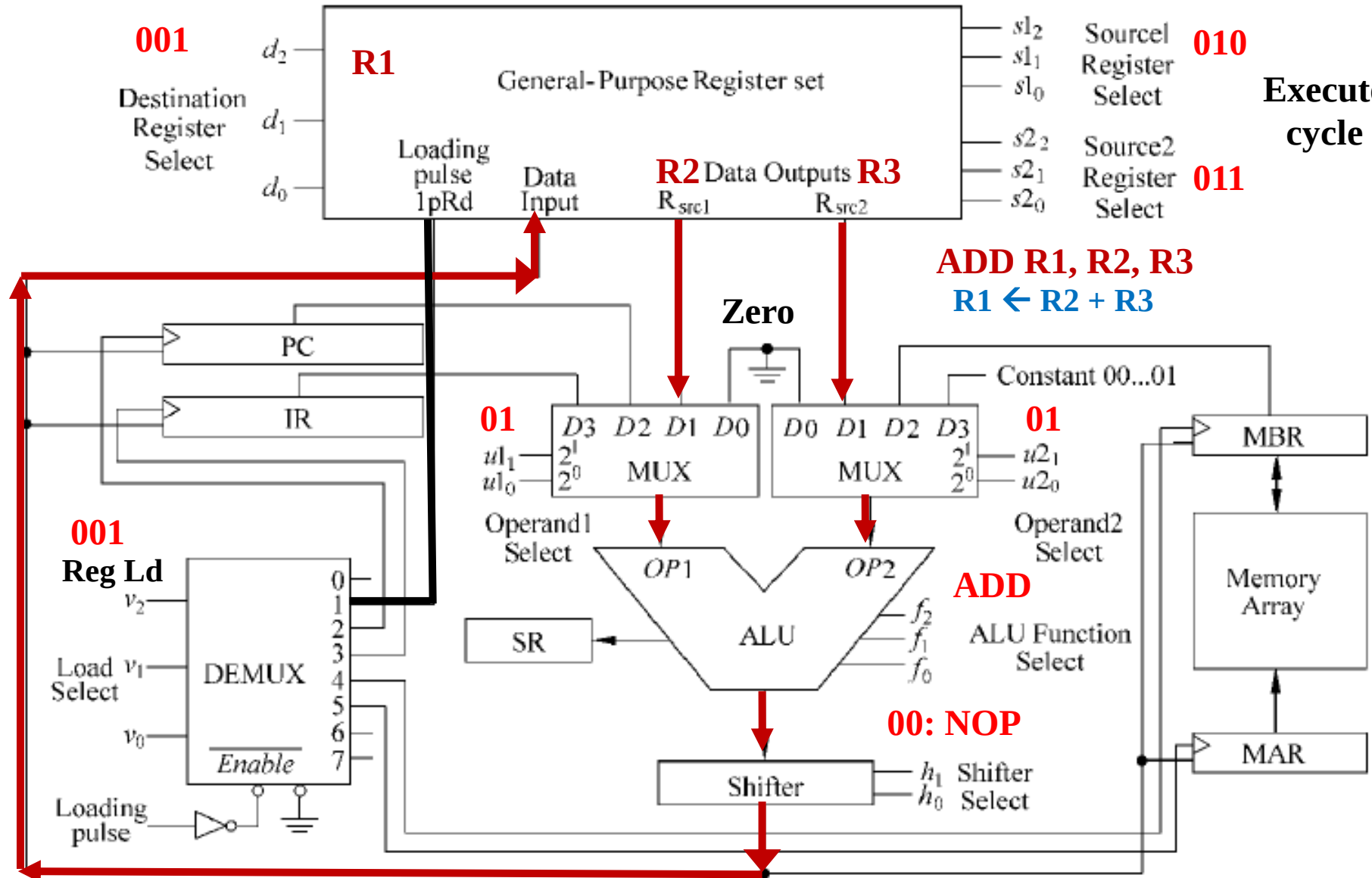
Control bus has the following signals:
RD/ $\overline{\text{WR}}$ commands **from CPU**
WAIT signal from the **memory**

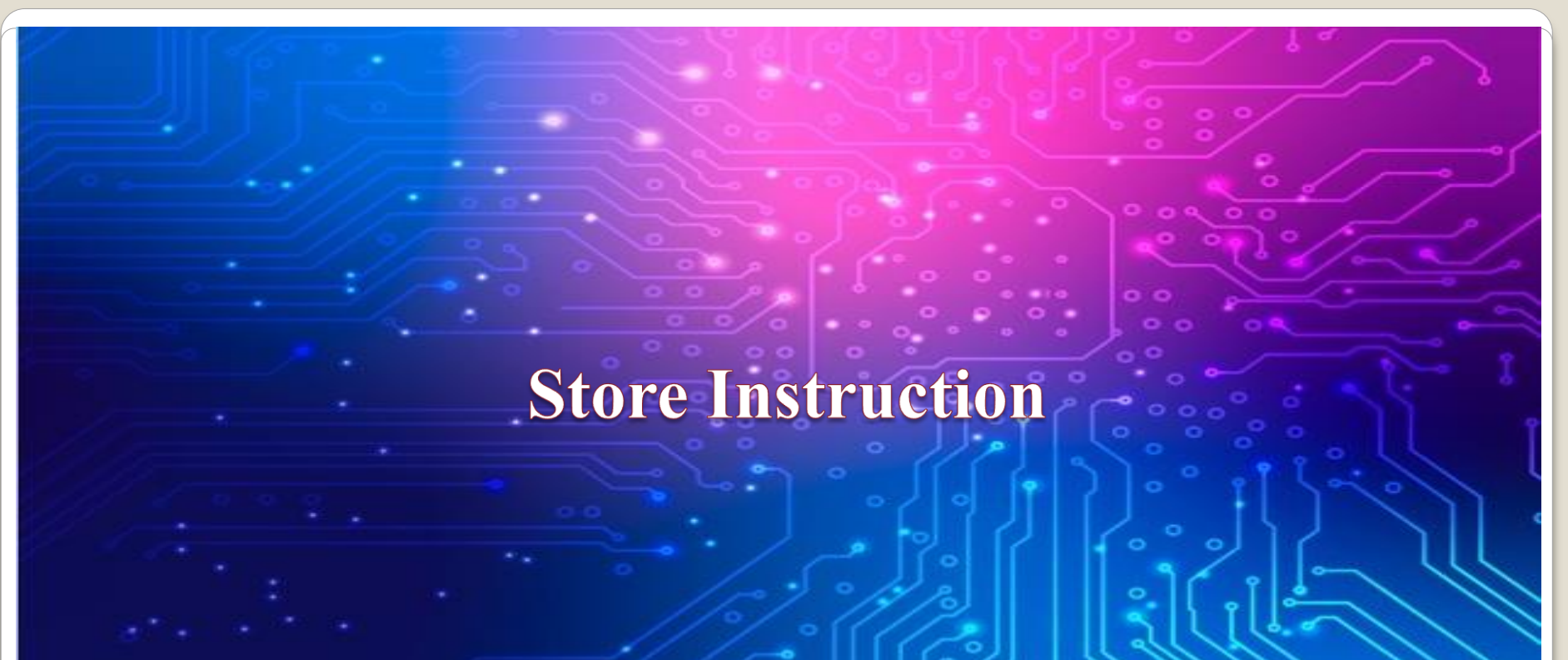


ADD Instruction

Instruction Execution: ADD R1, R2, R3

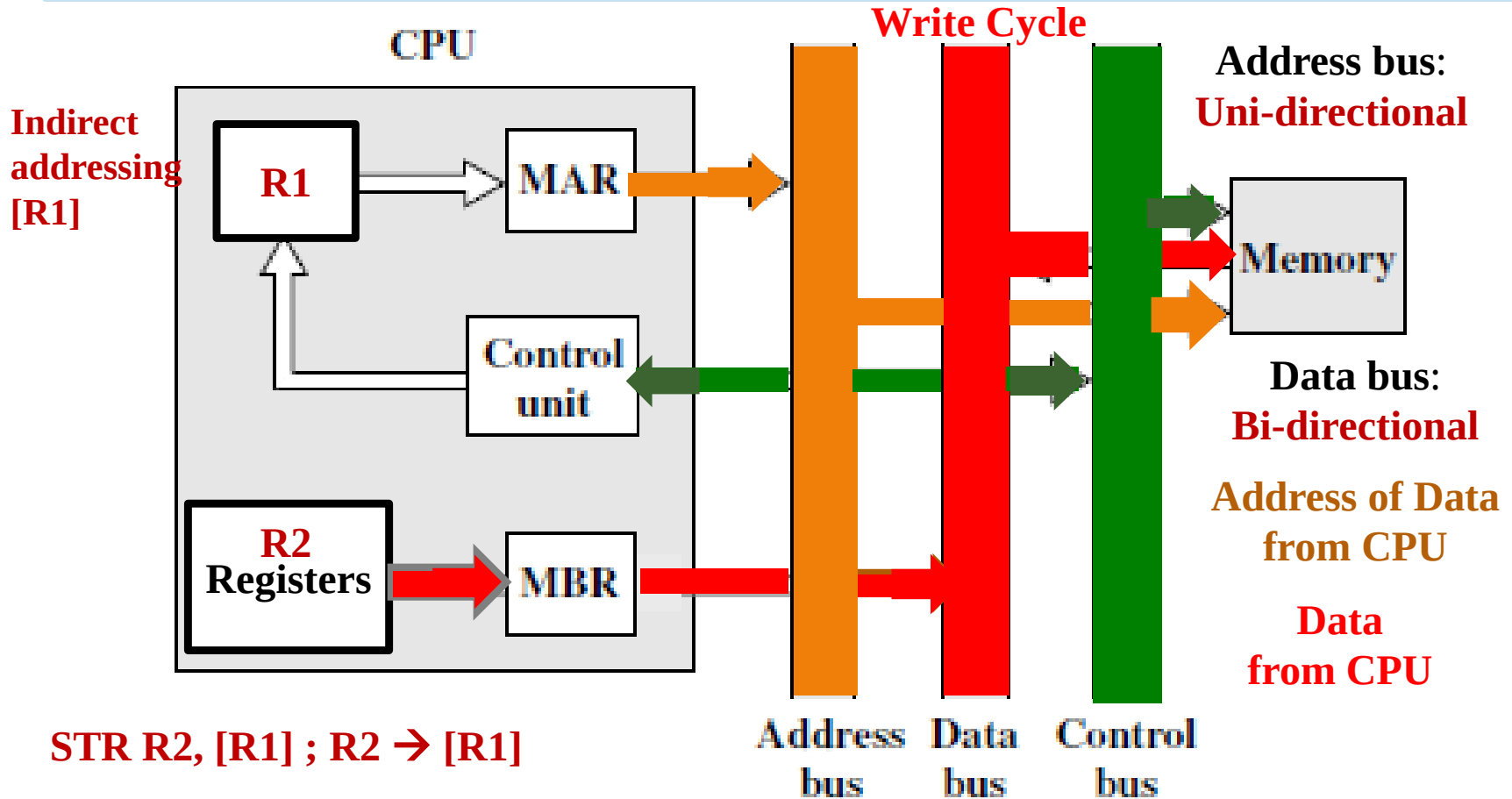
Execute cycle





Store Instruction

Store Instruction: Reg → Data Mem



MBR = Memory buffer register
MAR = Memory address register

Control bus has the following **signals**:
RD/WR commands **from CPU**
WAIT signal from the **memory**

Store Instruction: Reg → Data Mem

The assembly language instruction **STR R2, [R1]** is a store instruction, which is used to store the contents of register R2 into the memory location specified by the contents of register R1. Let's break down the instruction and explain how it works, including the interaction with the CPU and memory using the Memory Address Register (MAR) and Memory Buffer Register (MBR)

Instruction Fetch (IF):

The CPU fetches the instruction STR R2, [R1] from memory.

The program counter (PC) is incremented to point to the next instruction.

Decode (ID):

The CPU decodes the instruction to understand its operation.

In this case, the CPU recognizes that it is a store instruction.

Execution (EX):

The CPU calculates the effective address for the memory operation. In this case, it uses the contents of register R1 as the memory address.

The Memory Address Register (MAR) is loaded with the effective address (contents of R1).

Store Instruction: Reg → Data Mem

Memory Access (MEM):

The CPU initiates a memory read operation by placing the contents of the MAR onto the address bus. The memory reads the data from the specified address (contents of R1) and loads it into the Memory Buffer Register (MBR).

Write Back (WB):

The CPU stores the data from register R2 into the MBR.

The contents of R2 are placed onto the data bus.

The memory receives the data from the data bus and writes it to the address specified by the MAR (contents of R1)

MAR (Memory Address Register): Holds the memory address (contents of R1) during memory operations.

MBR (Memory Buffer Register): Holds the data read from memory (contents of the memory address specified by R1) and is later used to store the modified data before writing it back to the same memory address.